

Bardzo dokładne opracowanie pytań na egzamin dyplomowy

Informatyka i Systemy Informacyjne, specjalność Metody Sztucznej Inteligencji

Opracowanie przygotowane na podstawie listy pytań oraz materiałów w folderze projektu.

Spis treści

Jak korzystać z opracowania	8
Lista pytań objętych dokumentem	8
1 Podstawy teoretyczne, algorytmiczne i numeryczne	8
1.1 Hierarchia Chomsky’ego oraz odpowiadające jej gramatyki i automaty	8
1.1.1 Podstawowe pojęcia	8
1.1.2 Cztery klasy w hierarchii	8
1.1.3 Języki regularne	9
1.1.4 Języki bezkontekstowe	9
1.1.5 Języki kontekstowe	10
1.1.6 Języki typu 0	10
1.1.7 Co powiedzieć na obronie	10
1.2 Złożoność obliczeniowa algorytmu	10
1.2.1 Rozmiar wejścia i model obliczeń	10
1.2.2 Notacja asymptotyczna	10
1.2.3 Przypadek pesymistyczny, średni i amortyzowany	11
1.2.4 Typowe rzędy wzrostu	11
1.2.5 Złożoność pamięciowa	11
1.2.6 Co powiedzieć na obronie	11
1.3 Klasy problemów według złożoności. Problem P–NP	11
1.3.1 Klasa P	11
1.3.2 Klasa NP	12
1.3.3 Redukcje wielomianowe	12
1.3.4 Problemy NP-trudne i NP-zupełne	12
1.3.5 Problem P–NP	12
1.3.6 Inne klasy warte wspomnienia	13
1.3.7 Co powiedzieć na obronie	13
1.4 Testowanie i poszukiwanie błędów w implementacji metod inteligencji obliczeniowej wykorzystujących losowość	13
1.4.1 Reprodukowalność jako pierwszy warunek debugowania	13
1.4.2 Testy jednostkowe deterministycznych komponentów	13
1.4.3 Testy własności i niezmienników	14
1.4.4 Testy statystyczne losowości	14
1.4.5 Metamorphic testing	14
1.4.6 Debugowanie przebiegów optymalizacji	15
1.4.7 Porównanie z bazami odniesienia	15
1.4.8 Co powiedzieć na obronie	15
1.5 Metody prowadzenia eksperymentów obliczeniowych	15
1.5.1 Pytanie badawcze i hipotezy	16
1.5.2 Plan eksperymentu	16
1.5.3 Kontrola zmiennych	16
1.5.4 Powtarzalność i replikowalność	16
1.5.5 Metryki i analiza statystyczna	17
1.5.6 Ablation study i analiza czułości	17
1.5.7 Błędy w eksperymentach	17
1.5.8 Co powiedzieć na obronie	17
1.6 Problem komiwojażera: definicja i algorytmy rozwiązywania	17
1.6.1 Warianty problemu	18
1.6.2 Sformułowanie matematyczne	18
1.6.3 Algorytmy dokładne	18

1.6.4	Heurystyki konstrukcyjne	19
1.6.5	Ulepszanie lokalne	19
1.6.6	Aproksymacja dla metrycznego TSP	19
1.6.7	Metaheurystyki	19
1.6.8	Co powiedzieć na obronie	20
1.7	Zadania interpolacji i zastosowanie interpolacji	20
1.7.1	Interpolacja wielomianowa	20
1.7.2	Błąd interpolacji	20
1.7.3	Efekt Rungego i dobór węzłów	20
1.7.4	Splajny	21
1.7.5	Zastosowania interpolacji	21
1.7.6	Co powiedzieć na obronie	21
1.8	Metody skończone rozwiązywania układów równań liniowych	21
1.8.1	Eliminacja Gaussa	22
1.8.2	Wybór elementu głównego	22
1.8.3	Rozkład LU	22
1.8.4	Rozkład Cholesky'ego	22
1.8.5	Rozkład QR	22
1.8.6	Uwarunkowanie układu	23
1.8.7	Co powiedzieć na obronie	23
1.9	Metody poszukiwania zer funkcji jednej zmiennej	23
1.9.1	Metoda bisekcji	23
1.9.2	Metoda Newtona	23
1.9.3	Metoda siecznych	23
1.9.4	Reguła fałsi	24
1.9.5	Metody hybrydowe	24
1.9.6	Kryteria stopu	24
1.9.7	Co powiedzieć na obronie	24
1.10	Metody całkowania numerycznego	24
1.10.1	Złożone wzory Newtona-Cotesa	24
1.10.2	Kwadratury Gaussa	25
1.10.3	Metody adaptacyjne	25
1.10.4	Ekstrapolacja Richardsona i Romberg	25
1.10.5	Monte Carlo	25
1.10.6	Źródła błędów	26
1.10.7	Co powiedzieć na obronie	26
2	Programowanie matematyczne i algorytmy zaawansowane	26
2.1	Metody poszukiwania ekstremum funkcji nieliniowej	26
2.1.1	Warunki optymalności bez ograniczeń	26
2.1.2	Metoda największego spadku	27
2.1.3	Metoda Newtona	27
2.1.4	Metody quasi-Newtonowskie	27
2.1.5	Metody bezpochodne	28
2.1.6	Optymalizacja stochastyczna	28
2.1.7	Lokalne i globalne optimum	28
2.1.8	Co powiedzieć na obronie	28
2.2	Metody poszukiwania ekstremum w obecności ograniczeń	28
2.2.1	Warunki KKT	29
2.2.2	Metoda mnożników Lagrange'a	29
2.2.3	Metody kar i barier	29
2.2.4	Programowanie sekwencyjne kwadratowe	29

2.2.5	Metody projekcyjne	30
2.2.6	Active set i interior point	30
2.2.7	Ograniczenia całkowitoliczbowe	30
2.2.8	Co powiedzieć na obronie	30
2.3	Wielomianowy schemat aproksymacyjny	30
2.3.1	Współczynnik aproksymacji	31
2.3.2	PTAS	31
2.3.3	FPTAS	31
2.3.4	EPTAS	31
2.3.5	Przykłady	31
2.3.6	Schemat FPTAS dla plecaka	32
2.3.7	Co powiedzieć na obronie	32
2.4	Programowanie dynamiczne	32
2.4.1	Dwie podstawowe własności	32
2.4.2	Top-down i bottom-up	32
2.4.3	Schemat projektowania DP	32
2.4.4	Przykład: najdłuższy wspólny podciąg	33
2.4.5	Przykład: plecak 0/1	33
2.4.6	Przykład: Floyd-Warshall	33
2.4.7	Przykład: Held-Karp dla TSP	33
2.4.8	Pułapki	34
2.4.9	Co powiedzieć na obronie	34
3	Zarządzanie przedsięwzięciami informatycznymi	34
3.1	Czym charakteryzuje się projekt informatyczny?	34
3.1.1	Cechy projektu	34
3.1.2	Potrójne ograniczenie i jego rozszerzenia	35
3.1.3	Specyfika projektów informatycznych	35
3.1.4	Interesariusze	35
3.1.5	Zakres, wymagania i akceptacja	36
3.1.6	Ryzyka typowe dla projektów IT	36
3.1.7	Co powiedzieć na obronie	36
3.2	Główne różnice między projektem a pracą operacyjną	36
3.2.1	Porównanie	36
3.2.2	Relacja między projektem i operacjami	37
3.2.3	Przykład	37
3.2.4	Co powiedzieć na obronie	37
3.3	Metodyki zarządzania projektami: charakterystyka, różnice i cechy wspólne	37
3.3.1	Waterfall	37
3.3.2	PMBOK i podejście procesowe	38
3.3.3	PRINCE2	38
3.3.4	Agile	38
3.3.5	Scrum	39
3.3.6	Kanban	39
3.3.7	Extreme Programming	40
3.3.8	Lean	40
3.3.9	DevOps	40
3.3.10	Podejścia hybrydowe	40
3.3.11	Cechy wspólne metodyk	40
3.3.12	Najważniejsze różnice	41
3.3.13	Co powiedzieć na obronie	41

4	Reprezentacja wiedzy	41
4.1	Metoda rezolucji w rachunku predykatów	41
4.1.1	Idea dowodu przez sprzeczność	41
4.1.2	Literały i klauzule	42
4.1.3	Reguła rezolucji w rachunku zdań	42
4.1.4	Sprawdzenie formuł do postaci klauzulowej	42
4.1.5	Dodatkowe problemy w rachunku predykatów	42
4.1.6	Postać preneksowa	43
4.1.7	Skolemizacja	43
4.1.8	Unifikacja	44
4.1.9	Reguła rezolucji pierwszego rzędu	44
4.1.10	Procedura dowodzenia rezolucyjnego	44
4.1.11	Poprawność, pełność i problem strategii	45
4.1.12	Rezolucja SLD i Prolog	45
4.1.13	Co powiedzieć na obronie	45
4.2	Główne problemy w zagadnieniach wnioskowania o działaniach	45
4.2.1	Model dynamicznego świata	46
4.2.2	Ontologia działań	46
4.2.3	Frame problem	46
4.2.4	Ramification problem	47
4.2.5	Qualification problem	47
4.2.6	Problem skutków domyślnych	47
4.2.7	Niedeterminizm	48
4.2.8	Współbieżność	48
4.2.9	Czas, akcje trwające i scenariusze	48
4.2.10	Niepełna wiedza i obserwacje	48
4.2.11	Planowanie	49
4.2.12	Co powiedzieć na obronie	49
5	Metody sztucznej inteligencji i systemy ekspertowe	49
5.1	Sposób działania algorytmu ewolucyjnego	49
5.1.1	Elementy algorytmu	49
5.1.2	Ogólny schemat	50
5.1.3	Reprezentacja	50
5.1.4	Selekcja	50
5.1.5	Krzyżowanie i mutacja	51
5.1.6	Eksploracja i eksploatacja	51
5.1.7	Parametry	51
5.1.8	Co powiedzieć na obronie	52
5.2	Porównanie algorytmu ewolucyjnego z metodami optymalizacji w $\mathbb{R}^n$	52
5.2.1	Algorytm ewolucyjny	52
5.2.2	Metoda największego spadku	52
5.2.3	Metoda Neldera-Meada	52
5.2.4	Metody quasi-Newtonowskie	52
5.2.5	Porównanie jakości i sposobu działania	53
5.2.6	Kiedy wybrać którą metodę	53
5.2.7	Co powiedzieć na obronie	53
5.3	Metody symulacji Monte Carlo w grach	54
5.3.1	Podstawowa idea	54
5.3.2	Monte Carlo Tree Search	54
5.3.3	UCT	54
5.3.4	Zastosowania	54

5.3.5	Ulepszenia	55
5.3.6	Co powiedzieć na obronie	55
5.4	Pojęcie agenta w SI: opis formalny i własności	55
5.4.1	Agent i środowisko	55
5.4.2	Racjonalność	56
5.4.3	Typy agentów	56
5.4.4	Własności agentów	56
5.4.5	Agent w uczeniu ze wzmocnieniem	56
5.4.6	Co powiedzieć na obronie	56
5.5	Inteligencja rojowa: idea i przykłady realizacji	57
5.5.1	Cechy systemów rojowych	57
5.5.2	Ant Colony Optimization	57
5.5.3	Particle Swarm Optimization	57
5.5.4	Artificial Bee Colony	57
5.5.5	Boids i modele stadne	58
5.5.6	Zastosowania	58
5.5.7	Co powiedzieć na obronie	58
5.6	Programowanie w logice na przykładzie Prologu	58
5.6.1	Fakty, reguły i zapytania	58
5.6.2	Semantyka klauzul Horna	59
5.6.3	Unifikacja	59
5.6.4	Rezolucja SLD i backtracking	59
5.6.5	Rekursja	59
5.6.6	Negacja jako porażka	59
5.6.7	Cut	60
5.6.8	Co powiedzieć na obronie	60
5.7	Generowanie reguł minimalnych z bazy wiedzy w systemach eksperckich	60
5.7.1	System informacyjny i decyzyjny	60
5.7.2	Minimalność reguły	60
5.7.3	Podejście przez zbiory przybliżone	60
5.7.4	Macierz rozróżnialności	61
5.7.5	Miary reguł	61
5.7.6	Usuwanie redundancji	61
5.7.7	Związek z drzewami decyzyjnymi	61
5.7.8	Co powiedzieć na obronie	62
6	Podstawy przetwarzania danych	62
6.1	Metody redukcji wymiaru danych	62
6.1.1	Redukcja wymiaru a selekcja cech	62
6.1.2	PCA	62
6.1.3	SVD	63
6.1.4	LDA jako redukcja nadzorowana	63
6.1.5	Metody nieliniowe	63
6.1.6	Dobór liczby wymiarów	63
6.1.7	Co powiedzieć na obronie	63
6.2	Metody selekcji cech	63
6.2.1	Filtry	64
6.2.2	Wrappery	64
6.2.3	Metody wbudowane	64
6.2.4	Stability selection	64
6.2.5	Selekcja a przeciek danych	65
6.2.6	Co powiedzieć na obronie	65

6.3	Metody postępowania z brakami w danych	65
6.3.1	Mechanizmy braków	65
6.3.2	Usuwanie danych	65
6.3.3	Prosta imputacja	65
6.3.4	Imputacja modelowa	66
6.3.5	Indykatory braków	66
6.3.6	Modele obsługujące braki	66
6.3.7	Co powiedzieć na obronie	66
6.4	Estymatory błędów	66
6.4.1	Podział train/validation/test	66
6.4.2	Walidacja krzyżowa	67
6.4.3	Stratyfikacja i dane zależne	67
6.4.4	Bootstrap	67
6.4.5	Macierz pomyłek i miary klasyfikacji	67
6.4.6	Miary regresji	68
6.4.7	Bias i wariancja estymatora błędu	68
6.4.8	Co powiedzieć na obronie	68
7	Uczenie ze wzmocnieniem	68
7.1	Uczenie ze wzmocnieniem: pasywne, aktywne i polityki	68
7.1.1	Cykl interakcji	68
7.1.2	Polityka	68
7.1.3	Pasywne uczenie ze wzmocnieniem	69
7.1.4	Aktywne uczenie ze wzmocnieniem	69
7.1.5	On-policy i off-policy	69
7.1.6	Funkcje wartości	69
7.1.7	Co powiedzieć na obronie	70
7.2	Decyzyjny proces Markowa: definicja i przykłady	70
7.2.1	Definicja	70
7.2.2	Własność Markowa	70
7.2.3	Równania Bellmana	71
7.2.4	Przykłady MDP	71
7.2.5	Planowanie i uczenie	71
7.2.6	Co powiedzieć na obronie	71
7.3	Metoda różnic czasowych TD-learning: idea, zakres stosowania, wady i zalety oraz porównanie z Monte Carlo	72
7.3.1	TD(0)	72
7.3.2	Algorytm predykcji TD(0)	72
7.3.3	Porównanie TD i Monte Carlo	72
7.3.4	SARSA	73
7.3.5	Q-learning	73
7.3.6	Zakres stosowania	73
7.3.7	Wady	73
7.3.8	Zalety	74
7.3.9	Co powiedzieć na obronie	74
8	Sieci neuronowe	74
8.1	Uczenie nadzorowane i nienadzorowane: definicje i przykłady	74
8.1.1	Uczenie nadzorowane	74
8.1.2	Uczenie nienadzorowane	75
8.1.3	WTA i WTM	75
8.1.4	Porównanie	75

8.1.5	Co powiedzieć na obronie	76
8.2	Porównanie modeli sieciowych Hopfielda, Grossberga i Kohonena	76
8.2.1	Sieć Hopfielda	76
8.2.2	Modele Grossberga	76
8.2.3	Sieć Kohonena	77
8.2.4	Porównanie	77
8.2.5	Co powiedzieć na obronie	77
8.3	Dobór architektury sieci neuronowej do wybranego zadania	77
8.3.1	Klasyfikacja tabularna	77
8.3.2	Regresja	78
8.3.3	Obrazy	78
8.3.4	Sekwencje i szeregi czasowe	78
8.3.5	Tekst	78
8.3.6	Uczenie nienadzorowane i kompresja	78
8.3.7	Pamięć skojarzeniowa i odtwarzanie wzorców	78
8.3.8	Kryteria doboru rozmiaru	79
8.3.9	Walidacja architektury	79
8.3.10	Co powiedzieć na obronie	79
8.4	Idea i własności pamięci skojarzeniowych Hopfielda	79
8.4.1	Architektura	79
8.4.2	Uczenie Hebbowskie	80
8.4.3	Faza rozpoznawania	80
8.4.4	Funkcja energii	80
8.4.5	Atraktory i baseny przyciągania	80
8.4.6	Pojemność	80
8.4.7	Modyfikacje	81
8.4.8	Co powiedzieć na obronie	81
8.5	Algorytm propagacji wstecznej błędu, własności i podstawowe modyfikacje	81
8.5.1	Przejście w przód	81
8.5.2	Przejście wstecz	81
8.5.3	Tryby uczenia	82
8.5.4	Warunki stopu	82
8.5.5	Zalety backpropagation	82
8.5.6	Problemy	82
8.5.7	Momentum	83
8.5.8	Adaptacyjny learning rate	83
8.5.9	QuickProp i metody drugiego rzędu	83
8.5.10	Regularyzacja i stabilizacja	83
8.5.11	Co powiedzieć na obronie	84
8.6	Problem katastroficznego zapominania: na czym polega i jak przeciwdziałać	84
8.6.1	Dlaczego występuje	84
8.6.2	Stability-plasticity dilemma	84
8.6.3	Interleaved learning i rehearsal	84
8.6.4	Regularyzacja ważnych wag	85
8.6.5	Metody modularne	85
8.6.6	Zamrażanie i współdzielenie cech	85
8.6.7	Knowledge distillation	85
8.6.8	Architektury i reprezentacje	85
8.6.9	Ocena ciągłego uczenia	86
8.6.10	Co powiedzieć na obronie	86

Jak korzystać z opracowania

Dokument jest ułożony według pytań z listy, a nie według plików z wykładami. Przy każdym zagadnieniu najpierw podane są definicje i intuicja, potem formalizacja, typowe algorytmy albo twierdzenia, a na końcu elementy, które warto umieć powiedzieć ustnie na obronie.

Materiały lokalne wykorzystane bezpośrednio:

- Zalacznik-3-Lista-przykladowych-pytan-ISI-MSI-2020-12-16 copy.pdf - lista pytań.
- Slajdy/RW_01--RW_14.pdf - reprezentacja wiedzy, rezolucja, wnioskowanie o działaniach.
- merged.pdf - uczenie ze wzmocnieniem.
- wykłady_m_scalone.pdf - sieci neuronowe.

Lista pytań objętych dokumentem

Opracowanie obejmuje wszystkie zagadnienia z listy: hierarchię Chomsky'ego, złożoność obliczeniową, klasy problemów i P-NP, testowanie metod inteligencji obliczeniowej, eksperymenty obliczeniowe, TSP, interpolację, metody liniowe, zera funkcji, całkowanie numeryczne, programowanie matematyczne, algorytmy aproksymacyjne, programowanie dynamiczne, zarządzanie projektami IT, rezolucję, wnioskowanie o działaniach, algorytmy ewolucyjne, Monte Carlo w grach, agentów, inteligencję rojową, Prolog, reguły minimalne, redukcję wymiaru, selekcję cech, braki danych, estymatory błędów, uczenie ze wzmocnieniem, MDP, TD-learning oraz sieci neuronowe.

1 Podstawy teoretyczne, algorytmiczne i numeryczne

1.1 Hierarchia Chomsky'ego oraz odpowiadające jej gramatyki i automaty

Hierarchia Chomsky'ego porządkuje klasy języków formalnych według tego, jak skomplikowanych reguł gramatycznych potrzeba do ich opisu oraz jak silny model automatu jest potrzebny do ich rozpoznawania. Im wyższy poziom w hierarchii, tym większa ekspresyjność, ale też trudniejsze własności decyzyjne.

1.1.1 Podstawowe pojęcia

Alfabet Σ jest skończonym zbiorem symboli. Słowo to skończony ciąg symboli z Σ , a język to dowolny podzbiór Σ^* . Gramatyka formalna jest zwykle zapisywana jako

$$G = (N, \Sigma, P, S),$$

gdzie N jest zbiorem symboli nieterminalnych, Σ zbiorem terminali, P zbiorem produkcji, a $S \in N$ symbolem startowym. Produkcja mówi, jak wolno zastąpić fragment sentencji innym fragmentem. Język generowany przez gramatykę to zbiór słów terminalnych wyprowadzalnych z S .

Automat jest modelem rozpoznawania języka. Dla słowa wejściowego automat odpowiada, czy słowo należy do języka. Różne klasy automatów mają różną pamięć i różne możliwości sterowania obliczeniem.

1.1.2 Cztery klasy w hierarchii

Typ	Gramatyka	Ograniczenie produkcji	Odpowiadający automat
0	bez ograniczeń	$\alpha \rightarrow \beta$, gdzie α zawiera co najmniej jeden nieterminal	maszyna Turinga
1	kontekstowa	produkcyjne niemające długości, często $\alpha A \beta \rightarrow \alpha \gamma \beta$, $\gamma \neq \epsilon$	liniowo ograniczona maszyna Turinga
2	bezkontekstowa	$A \rightarrow \gamma$, po lewej dokładnie jeden nieterminal	automat ze stosem
3	regularna	np. $A \rightarrow aB$, $A \rightarrow a$, ewentualnie $A \rightarrow \epsilon$ w ograniczonej postaci	automat skończony

Zachodzą ścisłe inkluzje:

$$\mathcal{L}_3 \subsetneq \mathcal{L}_2 \subsetneq \mathcal{L}_1 \subsetneq \mathcal{L}_0.$$

Każdy język regularny jest bezkontekstowy, każdy bezkontekstowy jest kontekstowy, a każdy kontekstowy jest rekurencyjnie przeliczalny. Odwrotne zależności nie zachodzą.

1.1.3 Języki regularne

Języki regularne opisują najprostsze wzorce, bez pamięci o nieograniczonej strukturze zagnieżdżonej. Są równoważne:

- gramatykom regularnym,
- wyrażeniom regularnym,
- deterministycznym automatom skończonym,
- niedeterministycznym automatom skończonym.

Automat skończony ma skończoną liczbę stanów i nie ma stosu ani taśmy roboczej. Dlatego potrafi zapamiętać tylko skończoną informację o prefiksie wejścia. Typowy język regularny:

$$L = \{w \in \{0, 1\}^* : w \text{ kończy się na } 01\}.$$

Typowy język nieregularny:

$$L = \{a^n b^n : n \geq 0\}.$$

Nie jest regularny, bo wymaga porównania dwóch nieograniczonych licznosci.

1.1.4 Języki bezkontekstowe

Języki bezkontekstowe pozwalają opisywać strukturę rekurencyjną, np. nawiasowanie, drzewa składniowe i większość składni języków programowania. Produkcja ma postać $A \rightarrow \gamma$, więc nieterminal może być rozwijany niezależnie od kontekstu.

Automat ze stosem ma dodatkową pamięć LIFO. Dzięki temu rozpoznaje np.

$$\{a^n b^n : n \geq 0\}$$

przez odkładanie symboli dla liter a i zdejmowanie ich przy literach b . Nie rozpoznaje jednak w ogólności języków wymagających dwóch niezależnych liczników, np.

$$\{a^n b^n c^n : n \geq 0\}.$$

1.1.5 Języki kontekstowe

Gramatyki kontekstowe pozwalają na produkcje zależne od otoczenia symbolu. Warunek niemalejącej długości powoduje, że wyprowadzenie nie może dowolnie skracać sentencji. Odpowiada im liniowo ograniczona maszyna Turinga, czyli maszyna Turinga, która może korzystać tylko z pamięci liniowo ograniczonej rozmiarem wejścia. Przykładem języka kontekstowego niebędącego bezkontekstowym jest

$$\{a^n b^n c^n : n \geq 1\}.$$

1.1.6 Języki typu 0

Gramatyki typu 0 nie mają istotnych ograniczeń na produkcje poza tym, że lewa strona nie może być pusta. Odpowiadają maszynom Turinga i językom rekurencyjnie przeliczalnym: jeśli słowo należy do języka, maszyna kiedyś je zaakceptuje; jeśli nie należy, może odrzucić albo działać w nieskończoność. Ta klasa obejmuje więc problemy częściowo rozstrzygalne, ale niekoniecznie decyzyjne.

1.1.7 Co powiedzieć na obronie

Najważniejsza odpowiedź ustna: hierarchia Chomsky'ego klasyfikuje języki według ograniczeń reguł produkcji. Typ 3 to języki regularne i automaty skończone, typ 2 to języki bezkontekstowe i automaty ze stosem, typ 1 to języki kontekstowe i liniowo ograniczone maszyny Turinga, typ 0 to gramatyki bez ograniczeń i maszyny Turinga. Każda kolejna klasa jest ściśle szersza.

1.2 Złożoność obliczeniowa algorytmu

Złożoność obliczeniowa opisuje, jak zasoby potrzebne algorytmowi rosną wraz z rozmiarem danych wejściowych. Najczęściej analizuje się czas i pamięć, ale w praktyce mogą liczyć się też liczba zapytań do bazy, komunikacja sieciowa, zużycie energii albo liczba operacji arytmetycznych wysokiej precyzji.

1.2.1 Rozmiar wejścia i model obliczeń

Złożoność zawsze definiuje się względem parametru n , czyli rozmiaru wejścia. Dla tablicy może to być liczba elementów, dla grafu para $(|V|, |E|)$, dla liczby całkowitej liczba bitów jej zapisu, a nie sama wartość liczby. To rozróżnienie jest ważne: algorytm wielomianowy w wartości liczby może być wykładniczy w długości jej zapisu.

Model obliczeń określa, jakie operacje uznajemy za elementarne. W klasycznej analizie RAM zakłada się, że proste operacje arytmetyczne, porównania i dostęp do pamięci zajmują czas stały. W analizie bitowej koszt operacji zależy od długości operandów.

1.2.2 Notacja asymptotyczna

Dla funkcji kosztu $T(n)$ używa się:

- $T(n) = \mathcal{O}(g(n))$ - ograniczenie z góry z dokładnością do stałej,
- $T(n) = \Omega(g(n))$ - ograniczenie z dołu,
- $T(n) = \Theta(g(n))$ - dokładny rząd wzrostu,
- $T(n) = o(g(n))$ - wzrost istotnie wolniejszy,
- $T(n) = \omega(g(n))$ - wzrost istotnie szybszy.

Formalnie $T(n) = \mathcal{O}(g(n))$, gdy istnieją stałe $c > 0$ i n_0 , takie że dla każdego $n \geq n_0$ zachodzi $0 \leq T(n) \leq cg(n)$.

1.2.3 Przypadek pesymistyczny, średni i amortyzowany

Pesymistyczna złożoność mierzy najgorszy możliwy koszt dla wejść rozmiaru n . Daje gwarancję niezależną od danych.

Średnia złożoność wymaga rozkładu prawdopodobieństwa na wejściach. Jest użyteczna, ale zależy od założeń statystycznych.

Złożoność oczekiwana dotyczy zwykle algorytmów losowych i oczekiwania po losowości algorytmu.

Złożoność amortyzowana rozlicza koszt sekwencji operacji. Pojedyncza operacja może być droga, ale średni koszt w długiej sekwencji jest mały, np. dynamiczna tablica z podwajaniem rozmiaru.

1.2.4 Typowe rzędy wzrostu

Od najlepszych do najgorszych praktycznie:

$$\mathcal{O}(1), \quad \mathcal{O}(\log n), \quad \mathcal{O}(n), \quad \mathcal{O}(n \log n), \quad \mathcal{O}(n^2), \quad \mathcal{O}(n^3), \quad \mathcal{O}(2^n), \quad \mathcal{O}(n!), \quad \mathcal{O}(n^n).$$

Różnice asymptotyczne szybko dominują nad stałymi. Algorytm $\mathcal{O}(n \log n)$ dla dużych danych będzie zwykle jakościowo lepszy od $\mathcal{O}(n^2)$, nawet jeśli ma większą stałą.

1.2.5 Złożoność pamięciowa

Złożoność pamięciowa $S(n)$ mierzy dodatkową pamięć potrzebną algorytmowi. Często odróżnia się pamięć wejścia od pamięci roboczej. Algorytm sortowania przez scalanie ma czas $\Theta(n \log n)$ i typowo pamięć $\Theta(n)$, a heapsort czas $\Theta(n \log n)$ i pamięć dodatkową $\Theta(1)$.

1.2.6 Co powiedzieć na obronie

Złożoność algorytmu to funkcja opisująca zużycie zasobów w zależności od rozmiaru wejścia. Podajemy ją asymptotycznie, najczęściej w notacji $\mathcal{O}$, bo interesuje nas tempo wzrostu dla dużych danych. Trzeba wskazać model obliczeń, rozmiar wejścia i to, czy mówimy o czasie, pamięci, przypadku pesymistycznym, średnim czy amortyzowanym.

1.3 Klasy problemów według złożoności. Problem P–NP

Klasy złożoności grupują problemy decyzyjne według zasobów potrzebnych do ich rozwiązania. Problem decyzyjny ma odpowiedź TAK albo NIE. Wiele problemów optymalizacyjnych można badać przez wersje decyzyjne, np. zamiast "znajdź najkrótszą trasę TSP" pytamy "czy istnieje trasa o długości co najwyżej B ?".

1.3.1 Klasa P

Klasa P zawiera problemy decyzyjne rozwiązywalne deterministycznie w czasie wielomianowym względem długości wejścia:

$$P = \bigcup_{k \geq 1} \text{DTIME}(n^k).$$

Przykłady: osiągalność w grafie, najkrótsze ścieżki z nieujemnymi wagami w wersji decyzyjnej, sortowanie jako problem funkcyjny, sprawdzanie spójności grafu, maksymalny przepływ w wersji decyzyjnej.

Wielomianowość traktuje się jako formalny odpowiednik obliczalności efektywnej, choć praktycznie algorytm $\mathcal{O}(n^{100})$ nie jest użyteczny, a algorytm wykładniczy może działać dla małych n .

1.3.2 Klasa NP

Klasa NP zawiera problemy decyzyjne, dla których odpowiedź TAK ma certyfikat weryfikowalny deterministycznie w czasie wielomianowym. Równoważnie są to problemy rozwiązywalne przez niedeterministyczną maszynę Turinga w czasie wielomianowym.

Przykład: dla problemu Hamiltona certyfikatem jest kolejność wierzchołków. Weryfikacja, czy jest to cykl Hamiltona, jest wielomianowa. Dla TSP decyzyjnego certyfikatem jest permutacja miast; weryfikujemy, czy tworzy cykl i czy suma wag jest $\leq B$.

Zawsze $P \subseteq NP$, bo jeśli umiemy problem szybko rozwiązać, to umiemy też szybko zweryfikować certyfikat albo go zignorować.

1.3.3 Redukcje wielomianowe

Problem A redukuje się wielomianowo do problemu B , co zapisujemy $A \leq_p B$, jeśli istnieje funkcja obliczalna w czasie wielomianowym taka, że instancja x problemu A ma odpowiedź TAK wtedy i tylko wtedy, gdy $f(x)$ ma odpowiedź TAK w problemie B .

Redukcja oznacza: gdybyśmy umieli szybko rozwiązywać B , umielibyśmy szybko rozwiązywać A . Dlatego trudność przenosi się z A na B .

1.3.4 Problemy NP-trudne i NP-zupełne

Problem jest NP-trudny, jeśli każdy problem z NP redukuje się do niego w czasie wielomianowym. Problem jest NP-zupełny, jeśli jest NP-trudny i jednocześnie należy do NP.

Klasyczne problemy NP-zupełne:

- SAT i 3-SAT,
- CLIQUE w wersji decyzyjnej,
- VERTEX COVER,
- HAMILTONIAN CYCLE,
- TSP decyzyjny,
- SUBSET SUM.

Twierdzenie Cooka-Levina mówi, że SAT jest NP-zupełny. W praktyce wiele dowodów NP-zupełności polega na redukcji z problemu już znanego jako NP-zupełny.

1.3.5 Problem P–NP

Pytanie $P = NP$? brzmi: czy każdy problem, którego rozwiązanie można szybko zweryfikować, można też szybko rozwiązać? Wiadomo, że $P \subseteq NP$, ale nie wiadomo, czy inkluzja jest ścisła. Powszechne przekonanie brzmi $P \neq NP$, ponieważ przez dekady nie znaleziono algorytmów wielomianowych dla problemów NP-zupełnych mimo ogromnego wysiłku.

Jeżeli $P = NP$, wszystkie problemy NP-zupełne miałyby algorytmy wielomianowe. Miałoby to konsekwencje dla optymalizacji, automatycznego dowodzenia, kryptografii i wielu problemów planowania. Jeżeli $P \neq NP$, istnieją problemy łatwo weryfikowalne, ale niemożliwe do rozwiązania w czasie wielomianowym przez algorytm deterministyczny.

1.3.6 Inne klasy warte wspomnienia

co-NP problemy, których odpowiedzi NIE mają krótkie certyfikaty. Nie wiadomo, czy $NP = coNP$.

PSPACE problemy rozwiązywalne w pamięci wielomianowej. Obejmuje NP.

EXPTIME problemy rozwiązywalne w czasie wykładniczym.

BPP problemy rozwiązywalne przez algorytmy losowe w czasie wielomianowym z ograniczonym prawdopodobieństwem błędu.

RP, co-RP, ZPP klasy losowe z jednostronnym błędem albo zerowym błędem oczekiwanym.

1.3.7 Co powiedzieć na obronie

P to problemy decyzyjne rozwiązywalne w czasie wielomianowym. NP to problemy, dla których rozwiązanie można zweryfikować w czasie wielomianowym. Problem P–NP pyta, czy szybka weryfikacja oznacza szybkie rozwiązywanie. Problemy NP-zupełne są najtrudniejsze w NP: jeśli jeden z nich ma algorytm wielomianowy, to $P = NP$.

1.4 Testowanie i poszukiwanie błędów w implementacji metod inteligencji obliczeniowej wykorzystujących losowość

Metody inteligencji obliczeniowej często są niedeterministyczne: algorytmy ewolucyjne, symulowane wyżarzanie, Monte Carlo Tree Search, losowe inicjalizacje sieci neuronowych, dropout, losowe mini-batche, bagging albo metody próbkowania. Testowanie takich implementacji wymaga odróżnienia błędów programistycznych od naturalnej zmienności wyników.

1.4.1 Reprodukowalność jako pierwszy warunek debugowania

Podstawą jest kontrola generatorów liczb pseudolosowych. Należy:

- jawnie ustawiać ziarno generatora,
- zapisywać użyte ziarna razem z wynikami eksperymentu,
- kontrolować wszystkie źródła losowości, np. bibliotekę numeryczną, framework GPU, operacje wielowątkowe,
- umożliwić powtórzenie pojedynczego przebiegu bit po bicie, o ile środowisko na to pozwala.

Dobrym wzorcem jest rozdzielenie generatorów: osobny strumień dla inicjalizacji populacji, mutacji, selekcji, podziału danych, szumu eksploracyjnego. Ułatwia to znalezienie źródła rozbieżności.

1.4.2 Testy jednostkowe deterministycznych komponentów

Nawet metoda losowa składa się z wielu deterministycznych części. Testować należy osobno:

- funkcję celu i ograniczenia,

- kod dekodowania reprezentacji osobnika,
- obliczanie dopasowania,
- operatory krzyżowania i mutacji dla ustalonych wartości losowych,
- aktualizację wag, gradientów, wartości Q albo statystyk w drzewie MCTS,
- obsługę przypadków brzegowych: puste dane, jeden osobnik, zerowa wariancja, remisy, wartości NaN.

Warto wstrzykiwać generator losowy jako zależność. Wtedy test może użyć generatora deterministycznego zwracającego z góry znaną sekwencję.

1.4.3 Testy własności i niezmienników

W metodach inteligencji obliczeniowej często łatwiej sprawdzić własności niż konkretne wyniki:

- prawdopodobieństwa selekcji sumują się do 1,
- mutacja zachowuje dopuszczalny zakres genów,
- krzyżowanie permutacyjne zwraca permutację bez duplikatów,
- najlepsze znalezione rozwiązanie w algorytmie elitarnym nie pogarsza się między generacjami,
- operator projekcji zwraca punkt spełniający ograniczenia,
- funkcja straty maleje dla prostego problemu uczącego, jeżeli krok jest odpowiednio mały,
- estymator Monte Carlo jest nieobciążony na prostym rozkładzie testowym.

Takie testy można realizować jako property-based testing: generujemy wiele losowych wejść i sprawdzamy ogólne warunki.

1.4.4 Testy statystyczne losowości

Jeżeli komponent ma realizować określony rozkład, pojedynczy wynik niczego nie dowodzi. Sprawdza się zbiorcze statystyki:

- średnią i wariancję próbek,
- histogram kategorii,
- test χ^2 dla rozkładu dyskretnego,
- test Kołmogorowa-Smirnowa dla rozkładu ciągłego,
- przedziały ufności dla oczekiwanej jakości.

Przykład: jeśli mutacja wybiera każdy gen z prawdopodobieństwem p , to po wielu próbach liczba mutacji powinna mieścić się w sensownym przedziale wokół np . Test nie powinien być zbyt wąski, bo będzie fałszywie wykrywać błędy.

1.4.5 Metamorphic testing

Metamorphic testing polega na sprawdzaniu relacji między wynikami dla przekształconych danych wejściowych. Przykłady:

- przemnożenie funkcji celu przez dodatnią stałą nie powinno zmienić optimum,
- permutacja kolejności przykładów uczących nie powinna zmienić wyniku batchowego,

- dodanie stałej do wszystkich długości krawędzi w TSP może zmienić wartości tras, ale nie musi zmienić relacji, jeśli dodatek jest równy dla każdej trasy,
- powielenie identycznych próbek nie powinno zmieniać optimum empirycznego dla średniej straty.

1.4.6 Debugowanie przebiegów optymalizacji

Należy logować nie tylko wynik końcowy, ale też przebieg:

- najlepszą, średnią i najgorszą wartość funkcji celu w populacji,
- różnorodność populacji,
- rozkład długości kroków albo mutacji,
- temperaturę w symulowanym wyżarzaniu,
- stopę akceptacji ruchów,
- normy gradientów i wartości straty,
- liczbę wizyt i wartości w węzłach MCTS.

Typowe symptomy błędów: natychmiastowa utrata różnorodności, wartości NaN, brak jakiegokolwiek zmiany mimo mutacji, nierealnie szybka zbieżność, wyniki lepsze niż znane dolne ograniczenie, silna zależność od kolejności danych, brak powtarzalności mimo ustalonego ziarna.

1.4.7 Porównanie z bazami odniesienia

Implementację sprawdza się na:

- instancjach małych, dla których da się policzyć optimum brutalnie,
- problemach analitycznych z oczywistym optimum, np. funkcja sferyczna $f(x) = \sum_i x_i^2$,
- benchmarkach literaturowych,
- baseline'ach prostych, np. losowe przeszukiwanie, algorytm zachłanny, metoda lokalna.

Jeżeli złożona metoda regularnie przegrywa z losowym baseline'em na prostym zadaniu, zwykle oznacza to błąd implementacyjny, złe skalowanie parametrów albo niedopasowanie reprezentacji.

1.4.8 Co powiedzieć na obronie

Testując metody losowe, trzeba zapewnić reprodukowalność przez kontrolę ziaren, testować deterministyczne komponenty, sprawdzać niezmienniki i własności statystyczne, uruchamiać wiele niezależnych przebiegów oraz porównywać z prostymi benchmarkami. Wynik pojedynczego uruchomienia nie jest dowodem poprawności ani jakości.

1.5 Metody prowadzenia eksperymentów obliczeniowych

Eksperyment obliczeniowy ma odpowiedzieć na pytanie badawcze, a nie tylko wygenerować tabelę wyników. Wymaga hipotezy, kontrolowanego protokołu, rzetelnego doboru danych, powtarzalności i analizy statystycznej.

1.5.1 Pytanie badawcze i hipotezy

Najpierw definiuje się, co porównujemy i dlaczego. Przykłady:

- Czy metoda A daje mniejszy błąd klasyfikacji niż metoda B?
- Czy operator mutacji X poprawia jakość algorytmu ewolucyjnego na instancjach TSP?
- Jak rozmiar populacji wpływa na czas dojścia do rozwiązania?
- Czy metoda jest odporna na braki danych?

Hipoteza zerowa może mówić, że nie ma różnicy między metodami. Hipoteza alternatywna mówi, że różnica istnieje, np. metoda A ma mniejszą średnią stratę.

1.5.2 Plan eksperymentu

Dobry plan określa:

- dane lub instancje problemu,
- metody i ich implementacje,
- parametry i sposób ich strojenia,
- metryki jakości,
- budżet obliczeniowy,
- liczbę powtórzeń,
- sposób agregacji wyników,
- testy statystyczne,
- kryteria zakończenia.

Ważna zasada: zbiór testowy nie może służyć do strojenia parametrów. W uczeniu maszynowym typowy podział to train/validation/test albo walidacja krzyżowa z zewnętrzną pętlą testową.

1.5.3 Kontrola zmiennych

Aby porównanie było uczciwe, zmieniamy jedną rzecz naraz albo stosujemy formalny plan czynnikowy. Wszystkie metody powinny mieć porównywalny budżet: czas, liczba ewaluacji funkcji celu, liczba epok, liczba symulacji. W algorytmach optymalizacyjnych często bardziej uczciwe jest porównanie po liczbie wywołań funkcji celu niż po liczbie iteracji, bo jedna iteracja różnych metod może mieć różny koszt.

1.5.4 Powtarzalność i replikowalność

Powtarzalność oznacza możliwość uzyskania tych samych wyników w tym samym środowisku. Replikowalność oznacza potwierdzenie w innym środowisku albo przez inną implementację. Należy zapisywać:

- kod i wersję commita,
- wersje bibliotek,
- konfigurację sprzętu,
- parametry eksperymentu,

- ziarna losowości,
- surowe wyniki,
- skrypty do agregacji.

1.5.5 Metryki i analiza statystyczna

Metryka musi odpowiadać celowi. Dla klasyfikacji używa się accuracy, precision, recall, F1, AUC, log-loss; dla regresji MAE, MSE, RMSE, R^2 ; dla optymalizacji wartości funkcji celu, odchylenia od optimum, czasu, liczby ewaluacji, stabilności.

Dla metod losowych podaje się co najmniej średnią i odchylenie standardowe albo medianę i kwartyle. Przy rozkładach skośnych mediana bywa bardziej informacyjna niż średnia. Do porównań można użyć testu t dla sparowanych wyników, testu Wilcoxona, testu Friedmana z post-hoc dla wielu metod i wielu zbiorów danych. Sama różnica średnich bez niepewności jest słabym argumentem.

1.5.6 Ablation study i analiza czułości

Ablation study polega na usuwaniu albo podmienianiu składników metody, aby sprawdzić, które elementy naprawdę wpływają na wynik. Analiza czułości bada, jak wynik zmienia się przy zmianie parametrów. To ważne, bo metoda może działać dobrze tylko dla wąskiego zakresu parametrów, co zmniejsza jej praktyczną użyteczność.

1.5.7 Błędy w eksperymentach

Typowe błędy:

- przeciek danych między treningiem i testem,
- strojenie parametrów na zbiorze testowym,
- porównanie do słabego baseline'u,
- zbyt mała liczba powtórzeń,
- ignorowanie wariancji,
- wybiórcze raportowanie najlepszych wyników,
- inny budżet obliczeniowy dla różnych metod,
- brak kontroli losowości,
- wyciąganie wniosków z jednego datasetu albo jednej instancji.

1.5.8 Co powiedzieć na obronie

Eksperyment obliczeniowy powinien mieć jasną hipotezę, ustalony protokół, kontrolowane parametry, uczciwe baseline'y, powtórzenia, analizę statystyczną i możliwość odtworzenia. W metodach losowych trzeba raportować rozkład wyników, a nie tylko najlepszy przebieg.

1.6 Problem komiwożera: definicja i algorytmy rozwiązywania

Problem komiwożera, czyli TSP, jest jednym z najważniejszych problemów optymalizacji kombinatorycznej. Dane jest n miast i koszty przejazdu c_{ij} między parami miast. Należy znaleźć cykl Hamiltona o

minimalnym koszcie, czyli trasę odwiedzającą każde miasto dokładnie raz i wracającą do punktu startowego.

1.6.1 Warianty problemu

Symetryczny TSP $c_{ij} = c_{ji}$. Trasa nie zależy od kierunku przejścia krawędzi.

Asymetryczny TSP koszty mogą zależeć od kierunku, np. czas jazdy w mieście.

Metryczny TSP koszty spełniają nierówność trójkąta $c_{ij} \leq c_{ik} + c_{kj}$.

Euklidesowy TSP miasta są punktami w przestrzeni, a kosztem jest odległość euklidesowa.

Decyzyjny TSP pytanie brzmi, czy istnieje cykl o koszcie co najwyżej B .

Decyzyjna wersja TSP jest NP-zupełna, a wersja optymalizacyjna jest NP-trudna. Oznacza to, że nie oczekujemy algorytmu dokładnego wielomianowego dla ogólnego TSP, o ile $P \neq NP$.

1.6.2 Sformułowanie matematyczne

Dla symetrycznego TSP można wprowadzić zmienne binarne x_{ij} , gdzie $x_{ij} = 1$, jeśli krawędź (i, j) należy do trasy. Minimalizujemy

$$\min \sum_{i < j} c_{ij} x_{ij}.$$

Warunki stopnia:

$$\sum_{j \neq i} x_{ij} = 2 \quad \text{dla każdego miasta } i.$$

Same warunki stopnia dopuszczają jednak kilka rozłącznych cykli. Dlatego trzeba dodać ograniczenia eliminujące podcykle:

$$\sum_{i, j \in S, i < j} x_{ij} \leq |S| - 1 \quad \text{dla każdego } \emptyset \neq S \subsetneq V.$$

Tych ograniczeń jest wykładniczo wiele, ale w metodach branch-and-cut dodaje się tylko te naruszone.

1.6.3 Algorytmy dokładne

Brute force sprawdza wszystkie permutacje miast. Dla TSP symetrycznego liczba różnych tras to $(n - 1)!/2$. Koszt jest silnie wykładniczy.

Programowanie dynamiczne Held-Karpa definiuje $D[S, j]$ jako koszt najkrótszej ścieżki zaczynającej w mieście 1, odwiedzającej zbiór S i kończącej w j . Rekurencja:

$$D[S, j] = \min_{i \in S, i \neq j} (D[S \setminus \{j\}, i] + c_{ij}).$$

Złożoność czasu $\mathcal{O}(n^2 2^n)$ i pamięci $\mathcal{O}(n 2^n)$.

Branch and bound przeszukuje drzewo częściowych tras i odcina gałęzie, których dolne ograniczenie jest gorsze od najlepszego znanego rozwiązania.

Branch and cut łączy programowanie całkowitoliczbowe z generowaniem ograniczeń eliminujących podcykle i nierówności wzmacniających relaksację.

1.6.4 Heurystyki konstrukcyjne

Heurystyki szybko budują rozwiązanie, ale bez gwarancji optymalności:

- najbliższy sąsiad: z aktualnego miasta idziemy do najbliższego nieodwiedzonego,
- nearest insertion: wstawiamy kolejne miasto w miejsce najmniej zwiększające koszt,
- cheapest insertion: wybieramy globalnie najtańsze wstawienie,
- farthest insertion: najpierw wstawiamy miasta dalekie, żeby wcześniej ustalić strukturę trasy,
- algorytmy oparte na minimalnym drzewie rozpinającym.

1.6.5 Ulepszanie lokalne

Metody lokalne startują z dowolnej trasy i poprawiają ją przez ruchy lokalne:

2-opt usuwa dwie krawędzie i odwraca fragment trasy, jeśli to skraca cykl.

3-opt usuwa trzy krawędzie i rozważa kilka sposobów ponownego połączenia fragmentów.

Lin-Kernighan adaptacyjnie dobiera złożoność ruchu k -opt i jest bardzo skuteczny praktycznie.

2-opt jest prosty i często daje dużą poprawę. Warunek poprawy dla wymiany krawędzi (a, b) i (c, d) na (a, c) i (b, d) :

$$c_{ab} + c_{cd} > c_{ac} + c_{bd}.$$

1.6.6 Aproksymacja dla metrycznego TSP

Dla metrycznego TSP istnieją algorytmy z gwarancją:

- algorytm podwajania drzewa MST daje trasę nie gorszą niż $2 \cdot OPT$,
- algorytm Christofidesa daje gwarancję $1.5 \cdot OPT$ dla symetrycznego metrycznego TSP.

Christofides: budujemy MST, znajdujemy wierzchołki nieparzystego stopnia, liczymy minimalne skojarzenie doskonałe na nich, dodajemy je do MST, otrzymujemy multigraf eulerowski, przechodzimy cyklem Eulera i skracamy powtórzenia dzięki nierówności trójkąta.

1.6.7 Metaheurystyki

Dla dużych instancji stosuje się algorytmy przybliżone:

- symulowane wyżarzanie,
- tabu search,
- algorytmy genetyczne z reprezentacją permutacyjną,
- ant colony optimization,
- iterated local search,
- MCTS albo metody uczenia przez wzmocnianie w wariantach konstrukcyjnych.

Kluczowy problem reprezentacji: operator mutacji albo krzyżowania musi zachować permutację miast. Dlatego używa się np. PMX, OX, CX, swap mutation, inversion mutation.

1.6.8 Co powiedzieć na obronie

TSP polega na znalezieniu najkrótszego cyklu odwiedzającego każde miasto dokładnie raz. Jest NP-trudny, a wersja decyzyjna jest NP-zupełna. Dokładne metody to brute force, Held-Karp, branch and bound oraz programowanie całkowitoliczbowe. Praktycznie używa się heurystyk konstrukcyjnych, 2-opt/3-opt, Lin-Kernighana i metaheurystyk. Dla metrycznego TSP są algorytmy aproksymacyjne, np. Christofides z gwarancją $3/2$.

1.7 Zadania interpolacji i zastosowanie interpolacji

Interpolacja polega na wyznaczeniu funkcji, która przechodzi dokładnie przez dane punkty pomiarowe:

$$f(x_i) = y_i, \quad i = 0, \dots, n.$$

W odróżnieniu od aproksymacji, interpolacja wymaga dokładnego dopasowania w węzłach. Jest użyteczna, gdy wartości danych traktujemy jako dokładne albo chcemy skonstruować funkcję pomocniczą do obliczeń numerycznych.

1.7.1 Interpolacja wielomianowa

Dla $n + 1$ różnych węzłów $x_0, \dots, x_n$ istnieje dokładnie jeden wielomian p_n stopnia co najwyżej n , taki że $p_n(x_i) = y_i$. Można go zapisać w postaci Lagrange'a:

$$p_n(x) = \sum_{i=0}^n y_i L_i(x), \quad L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

Bazy L_i mają własność $L_i(x_j) = 1$ dla $i = j$ i 0 dla $i \neq j$.

Wzór Newtona używa ilorazów różnicowych:

$$p_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n \prod_{j=0}^{n-1} (x - x_j),$$

gdzie $a_k = f[x_0, \dots, x_k]$. Zaletą postaci Newtona jest łatwe dodanie nowego węzła bez przeliczania całego wielomianu.

1.7.2 Błąd interpolacji

Jeśli funkcja f ma pochodną rzędu $n + 1$, to dla interpolacji wielomianowej:

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

dla pewnego ξ zależnego od x . Błąd zależy więc od gładkości funkcji i rozmieszczenia węzłów.

1.7.3 Efekt Rungego i dobór węzłów

Interpolacja wielomianem wysokiego stopnia na równoodległych węzłach może silnie oscylować przy końcach przedziału. To efekt Rungego. Przeciwdziała się mu przez:

- użycie węzłów Czebyszewa,

- interpolację kawałkami,
- splajny,
- aproksymację zamiast interpolacji, jeśli dane są zaszumione.

Węzły Czebyszewa zagęszczają się przy końcach przedziału, zmniejszając maksymalny wpływ iloczynu $\prod_i (x - x_i)$.

1.7.4 Splajny

Splajn to funkcja sklejana z wielomianów niskiego stopnia, zwykle trzeciego. Splajn kubiczny na przedziałach $[x_i, x_{i+1}]$:

- interpoluje dane w węzłach,
- jest ciągły,
- ma ciągłą pierwszą i drugą pochodną,
- unika globalnych oscylacji typowych dla wielomianów wysokiego stopnia.

Warunki brzegowe mogą być naturalne ($S''(x_0) = S''(x_n) = 0$), z zadanymi pochodnymi albo okresowe.

1.7.5 Zastosowania interpolacji

- rekonstrukcja wartości między pomiarami,
- grafika komputerowa i animacja,
- przetwarzanie sygnałów i obrazów,
- numeryczne całkowanie i różniczkowanie,
- metoda elementów skończonych,
- tablice funkcji specjalnych,
- kalibracja czujników,
- trajektorie robotów.

1.7.6 Co powiedzieć na obronie

Interpolacja buduje funkcję przechodzącą przez zadane punkty. Najważniejsze metody to interpolacja wielomianowa Lagrange'a i Newtona oraz splajny. Trzeba znać istnienie i jednoznaczność wielomianu interpolacyjnego, wzór błędu oraz problem Rungego. W praktyce dla wielu punktów preferuje się splajny albo węzły Czebyszewa.

1.8 Metody skończone rozwiązywania układów równań liniowych

Układ równań liniowych zapisujemy jako

$$Ax = b,$$

gdzie $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$, a $x \in \mathbb{R}^n$ jest niewiadomą. Metody skończone, nazywane też bezpośrednimi, w arytmetyce dokładnej dają rozwiązanie po skończonej liczbie operacji. Odróżnia się je od metod iteracyjnych, które generują ciąg przybliżeń.

1.8.1 Eliminacja Gaussa

Eliminacja Gaussa sprowadza macierz do postaci trójkątnej górnej przez zerowanie elementów pod przekątną. Następnie rozwiązuje się układ przez podstawianie wsteczne.

Dla kroku k eliminujemy elementy a_{ik} dla $i > k$:

$$m_{ik} = \frac{a_{ik}}{a_{kk}}, \quad R_i \leftarrow R_i - m_{ik}R_k.$$

Złożoność czasowa wynosi $\Theta(n^3)$, a podstawianie wsteczne $\Theta(n^2)$.

1.8.2 Wybór elementu głównego

Bez pivotingu eliminacja może być niestabilna numerycznie albo niemożliwa, gdy pivot $a_{kk} = 0$. W częściowym wyborze elementu głównego zamieniamy wiersze tak, aby w kolumnie k wybrać największy co do modułu element wśród wierszy $k, \dots, n$. Poprawia to stabilność i w praktyce jest standardem.

Pełny pivoting pozwala zamieniać także kolumny, ale jest droższy i rzadziej potrzebny.

1.8.3 Rozkład LU

Eliminacja Gaussa odpowiada rozkładowi:

$$PA = LU,$$

gdzie P jest macierzą permutacji, L trójkątną dolną, a U trójkątną górną. Rozwiązanie $Ax = b$ przebiega wtedy:

$$Ly = Pb, \quad Ux = y.$$

Rozkład LU jest szczególnie opłacalny, gdy trzeba rozwiązać wiele układów z tą samą macierzą A i różnymi prawymi stronami b .

1.8.4 Rozkład Cholesky'ego

Jeśli A jest symetryczna i dodatnio określona, można użyć rozkładu:

$$A = LL^T.$$

Jest około dwa razy tańszy niż LU i stabilny dla tej klasy macierzy. Występuje często w metodzie najmniejszych kwadratów, procesach Gaussowskich, optymalizacji wypukłej i równaniach normalnych, choć same równania normalne mogą pogarszać uwarunkowanie.

1.8.5 Rozkład QR

Rozkład QR:

$$A = QR,$$

gdzie Q jest ortogonalna, a R trójkątna górna. Jest bardzo ważny w rozwiązywaniu problemów najmniejszych kwadratów:

$$\min_x \|Ax - b\|_2.$$

Ponieważ macierze ortogonalne nie zmieniają normy euklidesowej, QR jest numerycznie stabilniejszy niż równania normalne $A^T Ax = A^T b$.

1.8.6 Uwarunkowanie układu

Wrażliwość rozwiązania na zaburzenia danych opisuje liczba uwarunkowania:

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

Jeśli $\kappa(A)$ jest duża, mały błąd w A albo b może spowodować duży błąd w x . Stabilny algorytm nie naprawi złego uwarunkowania problemu, ale nie powinien znacząco dokładać własnego błędu.

1.8.7 Co powiedzieć na obronie

Metody skończone rozwiązują układ po skończonej liczbie operacji. Najważniejsza jest eliminacja Gaussa z pivotingiem oraz rozkłady LU, Cholesky'ego i QR. LU jest dobre dla wielu prawych stron, Cholesky dla macierzy symetrycznych dodatnio określonych, QR dla najmniejszych kwadratów i stabilności. Trzeba wspomnieć o koszcie $\Theta(n^3)$ i uwarunkowaniu.

1.9 Metody poszukiwania zer funkcji jednej zmiennej

Szukamy x^* takiego, że

$$f(x^*) = 0.$$

Metody różnią się wymaganiami wobec funkcji, szybkością zbieżności i gwarancjami.

1.9.1 Metoda bisekcji

Zakładamy, że f jest ciągła na $[a, b]$ i $f(a)f(b) < 0$. Z twierdzenia Darboux istnieje zero w przedziale. W każdym kroku bierzemy środek

$$c = \frac{a+b}{2}$$

i wybieramy połowę przedziału, w której następuje zmiana znaku.

Zalety: prostota, gwarancja zbieżności, odporność. Wady: zbieżność liniowa i potrzeba przedziału ze zmianą znaku. Po k krokach długość przedziału wynosi $(b-a)/2^k$, więc łatwo ustalić liczbę iteracji potrzebną dla tolerancji.

1.9.2 Metoda Newtona

Metoda Newtona korzysta z pochodnej i linearyzacji funkcji:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Interpretacja geometryczna: bierzemy styczną do wykresu w x_k i jej przecięcie z osią x .

Dla dobrego punktu startowego i prostego pierwiastka metoda ma zbieżność kwadratową. Może jednak zawieść, jeśli $f'(x_k)$ jest bliskie zero, punkt startowy jest zły, funkcja ma osobliwości albo iteracje opuszczają sensowny obszar.

1.9.3 Metoda siecznych

Metoda siecznych zastępuje pochodną ilorazem różnicowym:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Nie wymaga obliczania pochodnej. Zbieżność jest superliniowa, rzędu około 1.618, ale brak tak silnych gwarancji jak w bisekcji.

1.9.4 Regula falsi

Regula falsi także używa siecznej, ale zachowuje przedział, w którym funkcja zmienia znak. Dzięki temu ma gwarancję podobną do bisekcji, choć może zbiegać wolno, jeśli jeden koniec przedziału pozostaje długo nieruchomy. Istnieją modyfikacje, np. Illinois, poprawiające ten problem.

1.9.5 Metody hybrydowe

W praktyce często łączy się bezpieczeństwo bisekcji z szybkością Newtona albo siecznych. Przykładem jest metoda Brenta, która używa bisekcji, siecznych i interpolacji odwrotnej kwadratowej. Jest standardowym wyborem dla jednowymiarowego znajdowania pierwiastków, gdy mamy przedział ze zmianą znaku.

1.9.6 Kryteria stopu

Typowe kryteria:

- $|f(x_k)| < \varepsilon$,
- $|x_{k+1} - x_k| < \varepsilon$,
- długość przedziału $< \varepsilon$,
- przekroczenie maksymalnej liczby iteracji.

Samo małe $|f(x_k)|$ może być mylące dla funkcji bardzo płaskiej albo źle przeskalowanej, dlatego często łączy się kilka kryteriów.

1.9.7 Co powiedzieć na obronie

Bisekcja jest wolna, ale gwarantowana przy ciągłości i zmianie znaku. Newton jest bardzo szybki lokalnie, ale wymaga pochodnej i dobrego startu. Sieczne nie wymagają pochodnej i mają zbieżność superliniową. W praktyce dobre są metody hybrydowe, np. Brent, bo łączą gwarancję z szybkością.

1.10 Metody całkowania numerycznego

Całkowanie numeryczne, czyli kwadratura, przybliża

$$I = \int_a^b f(x) dx$$

za pomocą skończonej liczby wartości funkcji. Stosuje się je, gdy funkcji nie da się scałkować analitycznie, znamy ją tylko z pomiarów albo całka jest składnikiem większego algorytmu.

1.10.1 Złożone wzory Newtona-Cotesa

Najprostsze metody dzielą przedział na podprzedziały długości $h = (b - a)/n$.

Metoda prostokątów:

$$I \approx h \sum_{i=0}^{n-1} f(x_i)$$

albo wariant ze środkiem:

$$I \approx h \sum_{i=0}^{n-1} f\left(\frac{x_i + x_{i+1}}{2}\right).$$

Metoda trapezów:

$$I \approx h \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(x_i) \right).$$

Metoda Simpsona dla parzystego n :

$$I \approx \frac{h}{3} \left(f(x_0) + f(x_n) + 4 \sum_{\substack{i=1 \\ i \text{ nieparzyste}}}^{n-1} f(x_i) + 2 \sum_{\substack{i=2 \\ i \text{ parzyste}}}^{n-2} f(x_i) \right).$$

Dla funkcji dostatecznie gładkich złożona metoda trapezów ma błąd rzędu $\mathcal{O}(h^2)$, a metoda Simpsona $\mathcal{O}(h^4)$.

1.10.2 Kwadratury Gaussa

Kwadratury Gaussa dobierają węzły i wagi tak, aby całkować dokładnie wielomiany możliwie wysokiego stopnia:

$$\int_a^b f(x)w(x) dx \approx \sum_{i=1}^n A_i f(x_i).$$

Dla n węzłów kwadratura Gaussa jest dokładna dla wielomianów stopnia do $2n - 1$. Węzły są związane z pierwiastkami wielomianów ortogonalnych. Najczęstsza jest Gauss-Legendre dla wagi $w(x) = 1$ na $[-1, 1]$.

1.10.3 Metody adaptacyjne

Jeśli funkcja ma obszary trudne, np. ostre piki albo szybkie zmiany, równomierna siatka marnuje punkty tam, gdzie funkcja jest gładka. Metody adaptacyjne dzielą przedział tam, gdzie lokalny błąd jest duży. Adaptacyjny Simpson porównuje wynik na całym przedziale z sumą wyników na połowach i rekurencyjnie zagęszcza podział.

1.10.4 Ekstrapolacja Richardsona i Romberg

Jeśli znamy rozwinięcie błędu metody, można połączyć wyniki dla kroków h i $h/2$, aby wyeliminować główny składnik błędu. Metoda Romberga stosuje ekstrapolację Richardsona do metody trapezów, tworząc tablicę coraz dokładniejszych przybliżeń.

1.10.5 Monte Carlo

Dla całek wielowymiarowych metody deterministyczne cierpią na przekleństwo wymiarowości. Monte Carlo przybliża wartość oczekiwaną:

$$\int_D f(x) dx = |D| \mathbb{E}[f(X)],$$

gdzie X jest losowane jednostajnie z D . Estymator:

$$\hat{I} = \frac{|D|}{N} \sum_{i=1}^N f(X_i)$$

ma błąd standardowy malejący jak $\mathcal{O}(N^{-1/2})$, niezależnie od wymiaru w sensie tempa względem liczby próbek. Wadą jest wolna zbieżność i losowa zmienność.

1.10.6 Źródła błędów

Błąd całkowania obejmuje:

- błąd obcięcia wynikający z przybliżenia całki sumą,
- błąd zaokrągleń arytmetyki zmiennoprzecinkowej,
- błędy danych wejściowych,
- trudności przy osobliwościach, nieciągłościach i funkcjach oscylacyjnych.

1.10.7 Co powiedzieć na obronie

Podstawowe metody to prostokąty, trapezy i Simpson, dalej kwadratury Gaussa, metody adaptacyjne, Romberg oraz Monte Carlo. Simpson jest dokładniejszy od trapezów dla funkcji gładkich, Gauss wykorzystuje optymalny dobór węzłów, metody adaptacyjne zagęszczają siatkę tam, gdzie funkcja jest trudna, a Monte Carlo jest szczególnie ważne w dużym wymiarze.

2 Programowanie matematyczne i algorytmy zaawansowane

2.1 Metody poszukiwania ekstremum funkcji nieliniowej

Rozważamy problem

$$\min_{x \in \mathbb{R}^n} f(x)$$

albo równoważnie maksymalizację przez minimalizację $-f(x)$. Funkcja może być wypukła albo niewypukła, gładka albo niegładka, tania albo kosztowna w ewaluacji. Wybór metody zależy głównie od dostępności pochodnych, wymiaru, wypukłości, szumu i kosztu obliczenia funkcji.

2.1.1 Warunki optymalności bez ograniczeń

Jeśli f jest różniczkowalna i x^* jest lokalnym minimum wewnętrznym, to konieczny warunek pierwszego rzędu:

$$\nabla f(x^*) = 0.$$

Jeśli f jest dwa razy różniczkowalna, konieczny warunek drugiego rzędu:

$$\nabla^2 f(x^*) \succeq 0.$$

Warunek wystarczający dla ścisłego minimum lokalnego:

$$\nabla f(x^*) = 0, \quad \nabla^2 f(x^*) \succ 0.$$

Dla funkcji wypukłej każdy punkt spełniający $\nabla f(x^*) = 0$ jest minimum globalnym. Dla funkcji ściśle wypukłej minimum globalne jest co najwyżej jedno.

2.1.2 Metoda największego spadku

Metoda największego spadku, czyli gradient descent, wykonuje kroki:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k),$$

gdzie $\alpha_k > 0$ jest długością kroku. Kierunek $-\nabla f(x_k)$ jest kierunkiem najszybszego lokalnego spadku w normie euklidesowej.

Długość kroku można dobrać:

- jako stałą,
- przez dokładne przeszukiwanie liniowe,
- przez warunki Armijo albo Wolfe'a,
- według harmonogramu malejącego, zwłaszcza w metodach stochastycznych.

Dla funkcji wypukłej z gradientem Lipschitza, przy odpowiednim kroku metoda zbiega do minimum. Dla funkcji silnie wypukłej zbieżność jest liniowa, ale może być bardzo wolna przy złym uwarunkowaniu, gdy poziomice są wydłużone.

2.1.3 Metoda Newtona

Metoda Newtona używa przybliżenia kwadratowego:

$$f(x_k + p) \approx f(x_k) + \nabla f(x_k)^T p + \frac{1}{2} p^T \nabla^2 f(x_k) p.$$

Kierunek Newtona spełnia:

$$\nabla^2 f(x_k) p_k = -\nabla f(x_k),$$

a aktualizacja:

$$x_{k+1} = x_k + \alpha_k p_k.$$

Jeśli Hessian jest dodatnio określony i punkt startowy jest dostatecznie blisko minimum, metoda ma zbieżność kwadratową. Wady: koszt obliczenia i odwracania/rozwiązywania układu z Hessianem, problemy przy Hessianie osobliwym albo nieokreślonym oraz brak globalnych gwarancji bez line search albo trust region.

2.1.4 Metody quasi-Newtonowskie

Metody quasi-Newtonowskie przybliżają Hessian albo jego odwrotność na podstawie kolejnych gradientów. Najbardziej znana jest BFGS. Niech

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

BFGS aktualizuje przybliżenie Hessianu tak, aby spełniało równanie secant:

$$B_{k+1} s_k = y_k.$$

L-BFGS przechowuje tylko kilka ostatnich par (s_k, y_k) , dlatego nadaje się do dużych wymiarów.

Zalety: zwykle szybsze niż gradient descent, bez jawnego Hessianu. Wady: wymagają gradientu, a dla funkcji silnie niewypukłych lub zaszumionych mogą wymagać zabezpieczeń.

2.1.5 Metody bezpochodne

Gdy gradient nie jest dostępny, funkcja jest szumowa albo niedokładna, stosuje się metody bezpochodne:

Nelder-Mead utrzymuje sympleks $n + 1$ punktów i wykonuje odbicie, ekspansję, kontrakcję i redukcję. Działa dobrze w małych wymiarach, ale nie ma silnych gwarancji w ogólności.

Pattern search bada skończony zbiór kierunków i zmniejsza krok, gdy nie ma poprawy.

CMA-ES adaptuje rozkład normalny próbkowania i macierz kowariancji. Jest silny dla trudnych, niewypukłych problemów ciągłych, ale kosztowny.

Symulowane wyżarzanie akceptuje czasem gorsze ruchy z prawdopodobieństwem malejącym wraz z temperaturą, co pomaga wyjść z minimów lokalnych.

2.1.6 Optymalizacja stochastyczna

Dla funkcji będącej średnią po danych:

$$f(x) = \frac{1}{m} \sum_{i=1}^m \ell_i(x)$$

pełny gradient może być kosztowny. Stochastic Gradient Descent używa losowej próbki albo mini-batcha:

$$x_{k+1} = x_k - \alpha_k g_k, \quad \mathbb{E}[g_k] = \nabla f(x_k).$$

W uczeniu maszynowym popularne są modyfikacje z momentum, RMSProp, Adam i AdamW. Ich zaletą jest skalowalność, wadą wrażliwość na hiperparametry i trudniejsza analiza końcowej dokładności.

2.1.7 Lokalne i globalne optimum

Dla funkcji niewypukłych metoda lokalna zwykle gwarantuje co najwyżej znalezienie punktu stacjonarnego albo lokalnego minimum. Globalna optymalizacja wymaga dodatkowych technik: wielostartu, branch and bound, relaksacji wypukłych, metaheurystyk, metod przedziałowych albo specjalnej struktury problemu.

2.1.8 Co powiedzieć na obronie

Najważniejsze metody bez ograniczeń to gradient descent, Newton, quasi-Newton, metody bezpochodne i stochastyczne. Gradient descent jest prosty, ale może być wolny; Newton jest szybki lokalnie, ale kosztowny; BFGS/L-BFGS to praktyczny kompromis; Nelder-Mead i inne metody bezpochodne są użyteczne, gdy nie mamy gradientu. W problemach wypukłych optimum lokalne jest globalne, w niewypukłych nie.

2.2 Metody poszukiwania ekstremum w obecności ograniczeń

Problem z ograniczeniami ma postać:

$$\min_x f(x)$$

przy warunkach

$$g_i(x) \leq 0, \quad i = 1, \dots, m, \quad h_j(x) = 0, \quad j = 1, \dots, p.$$

Zbiór dopuszczalny jest zbiorem punktów spełniających ograniczenia. Ograniczenia mogą być liniowe, nieliniowe, wypukłe, niewypukłe, równościowe, nierównościowe albo całkowitoliczbowe.

2.2.1 Warunki KKT

Dla problemu gładkiego definiujemy Lagrangian:

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{j=1}^p \mu_j h_j(x),$$

gdzie $\lambda_i \geq 0$ są mnożnikami dla nierówności, a μ_j dla równości.

Warunki Karusha-Kuhna-Tuckera:

$\nabla_x \mathcal{L}(x^*, \lambda^*, \mu^*) = 0$	stacjonarność,
$g_i(x^*) \leq 0, \quad h_j(x^*) = 0$	dopuszczalność pierwotna,
$\lambda_i^* \geq 0$	dopuszczalność dualna,
$\lambda_i^* g_i(x^*) = 0$	komplementarność.

Dla problemów wypukłych przy odpowiednich warunkach regularności KKT są nie tylko konieczne, ale też wystarczające.

2.2.2 Metoda mnożników Lagrange'a

Dla samych równości $h_j(x) = 0$ szukamy punktów stacjonarnych Lagrangianu:

$$\nabla f(x^*) + \sum_{j=1}^p \mu_j \nabla h_j(x^*) = 0, \quad h_j(x^*) = 0.$$

Geometrycznie gradient funkcji celu w optimum leży w rozpięciu gradientów aktywnych ograniczeń, czyli nie ma składowej pozwalającej poprawić wynik wzdłuż zbioru dopuszczalnego.

2.2.3 Metody kar i barier

Metody kar zamieniają problem ograniczony na sekwencję problemów bez ograniczeń:

$$\min_x f(x) + \rho_k P(x),$$

gdzie $P(x)$ karze naruszenie ograniczeń, np.

$$P(x) = \sum_i \max(0, g_i(x))^2 + \sum_j h_j(x)^2.$$

Gdy ρ_k rośnie, rozwiązania powinny zbliżać się do dopuszczalnych. Wadą jest pogarszające się uwarunkowanie dla dużych kar.

Metody barier działają wewnątrz zbioru dopuszczalnego, np. dla $g_i(x) < 0$:

$$\min_x f(x) - \mu \sum_i \log(-g_i(x)).$$

Gdy $\mu \rightarrow 0$, bariera słabnie, a rozwiązanie zbliża się do optimum problemu oryginalnego. To idea metod punktu wewnętrznego.

2.2.4 Programowanie sekwencyjne kwadratowe

Sequential Quadratic Programming rozwiązuje w każdej iteracji przybliżony problem kwadratowy: funkcję celu zastępuje modelem kwadratowym Lagrangianu, a ograniczenia linearyzuje. SQP jest bardzo skuteczne dla gładkich problemów nieliniowych średniego rozmiaru.

2.2.5 Metody projekcyjne

Dla prostego zbioru dopuszczalnego C można wykonać krok gradientowy i projekcję:

$$x_{k+1} = P_C(x_k - \alpha_k \nabla f(x_k)),$$

gdzie

$$P_C(y) = \arg \min_{x \in C} \|x - y\|_2.$$

Metoda jest naturalna dla ograniczeń pudełkowych, kul, sympleksu albo prostych zbiorów wypukłych. W uczeniu maszynowym odpowiada np. obcinaniu wag do zakresu albo projekcji na ograniczenie normy.

2.2.6 Active set i interior point

Metody aktywnego zbioru zgadują, które ograniczenia nierównościowe są aktywne w optimum, czyli spełnione jako równości. Następnie rozwiązują problem z tymi ograniczeniami jak z równościami i aktualizują zbiór aktywny.

Metody punktu wewnętrznego poruszają się we wnętrzu zbioru dopuszczalnego i używają barier. Są podstawą bardzo wydajnych solverów dla programowania liniowego, kwadratowego i wypukłego.

2.2.7 Ograniczenia całkowitoliczbowe

Jeśli część zmiennych musi być całkowita albo binarna, problem staje się programowaniem całkowitoliczbowym. Typowe techniki:

- branch and bound,
- branch and cut,
- relaksacja liniowa albo wypukła,
- płaszczyzny tnące,
- heurystyki naprawcze.

Tego typu problemy są często NP-trudne.

2.2.8 Co powiedzieć na obronie

Dla ograniczeń kluczowe są warunki KKT: stacjonarność, dopuszczalność, nieujemność mnożników i komplementarność. Metody praktyczne to mnożniki Lagrange'a, kary, bariery, projekcja, SQP, active set i interior point. Dla problemów wypukłych warunki KKT dają globalną charakterystykę optimum; dla niewypukłych zwykle tylko lokalną.

2.3 Wielomianowy schemat aproksymacyjny

Wiele problemów NP-trudnych jest zbyt kosztownych do dokładnego rozwiązania dla dużych instancji. Algorytmy aproksymacyjne zwracają rozwiązania bliskie optymalnym z formalną gwarancją jakości.

2.3.1 Współczynnik aproksymacji

Dla problemu minimalizacyjnego algorytm ma współczynnik $\rho(n) \geq 1$, jeśli dla każdej instancji:

$$A(I) \leq \rho(n) \cdot OPT(I),$$

gdzie $A(I)$ jest wartością rozwiązania algorytmu, a $OPT(I)$ optimum. Dla problemu maksymalizacyjnego zwykle wymaga się:

$$A(I) \geq \frac{1}{\rho(n)} OPT(I).$$

Jeśli ρ jest stałe, mówimy o stałej aproksymacji.

2.3.2 PTAS

Polynomial-Time Approximation Scheme to rodzina algorytmów, która dla dowolnego $\varepsilon > 0$ zwraca rozwiązanie o jakości:

$$A(I) \leq (1 + \varepsilon) OPT(I)$$

dla minimalizacji albo

$$A(I) \geq (1 - \varepsilon) OPT(I)$$

dla maksymalizacji, w czasie wielomianowym względem rozmiaru wejścia n dla ustalonego ε .

Istotne: czas może być bardzo zły jako funkcja $1/\varepsilon$, np.

$$\mathcal{O}(n^{1/\varepsilon}).$$

Dla każdego ustalonego ε jest to wielomian w n , więc formalnie PTAS.

2.3.3 FPTAS

Fully Polynomial-Time Approximation Scheme wymaga czasu wielomianowego zarówno względem n , jak i $1/\varepsilon$, np.

$$\mathcal{O}(n^3/\varepsilon^2).$$

FPTAS jest znacznie silniejszy praktycznie i teoretycznie niż PTAS.

2.3.4 EPTAS

Efficient PTAS ma czas postaci

$$f(1/\varepsilon) \cdot n^{O(1)},$$

gdzie wykładnik przy n nie zależy od ε . To kompromis między PTAS i FPTAS.

2.3.5 Przykłady

- Problem plecakowy 0/1 ma FPTAS oparty na skalowaniu wartości przedmiotów i programowaniu dynamicznym.
- Euklidesowy TSP w ustalonym wymiarze ma PTAS.
- Dla ogólnego metrycznego TSP znany jest algorytm Christofidesa z gwarancją $3/2$, ale nie jest to PTAS.
- Dla wielu problemów silnie NP-trudnych istnienie FPTAS implikowałoby $P = NP$.

2.3.6 Schemat FPTAS dla plecaka

W plecaku maksymalizujemy wartość przy ograniczeniu wagi. Klasyczne DP po wartościach działa w czasie zależnym od sumy wartości, która może być pseudowielomianowa. FPTAS skaluje wartości:

$$v'_i = \left\lfloor \frac{v_i}{K} \right\rfloor$$

dla odpowiednio dobranego K zależnego od ε i $v_{\max}$. Następnie rozwiązujemy problem DP dla mniejszych wartości. Tracimy co najwyżej kontrolowany ułamek optimum.

2.3.7 Co powiedzieć na obronie

PTAS to rodzina algorytmów, która dla dowolnej dokładności ε daje rozwiązanie w granicach $1 + \varepsilon$ od optimum w czasie wielomianowym dla ustalonego ε . FPTAS jest silniejszy, bo czas jest wielomianowy także względem $1/\varepsilon$. Schematy aproksymacyjne są sposobem radzenia sobie z NP-trudnością, gdy optimum dokładne jest za drogie.

2.4 Programowanie dynamiczne

Programowanie dynamiczne jest techniką projektowania algorytmów dla problemów z optymalną podstrukturą i nakładającymi się podproblemami. Zamiast wielokrotnie rozwiązywać te same podproblemy, zapamiętuje się ich wyniki.

2.4.1 Dwie podstawowe własności

Optymalna podstruktura rozwiązanie optymalne problemu można zbudować z rozwiązań optymalnych podproblemów.

Nakładające się podproblemy rekurencyjne rozwiązanie generowałoby te same podproblemy wielokrotnie.

Jeśli podproblemy się nie nakładają, zwykle wystarcza dziel i zwyciężaj, jak w mergesort. Jeśli nie ma optymalnej podstruktury, DP może dawać błędne rozwiązania.

2.4.2 Top-down i bottom-up

Podejście top-down z memoizacją zapisuje wyniki funkcji rekurencyjnej. Jest łatwe do napisania i oblicza tylko potrzebne stany.

Podejście bottom-up wypełnia tablicę w porządku zgodnym z zależnościami. Zwykle daje mniejszy narzut i łatwiej kontrolować pamięć.

2.4.3 Schemat projektowania DP

1. Zdefiniować stan, czyli co opisuje podproblem.
2. Określić wartość przechowywaną w stanie.
3. Wyprowadzić rekurencję przejścia.
4. Ustalić przypadki bazowe.
5. Ustalić kolejność obliczeń.

6. Odtworzyć rozwiązanie, jeśli potrzebna jest struktura, a nie tylko wartość.
7. Oszacować liczbę stanów i koszt przejścia.

2.4.4 Przykład: najdłuższy wspólny podciąg

Dla napisów $X[1..m]$ i $Y[1..n]$ definiujemy $dp[i][j]$ jako długość LCS prefiksów $X[1..i]$ i $Y[1..j]$:

$$dp[i][j] = \begin{cases} 0, & i = 0 \text{ lub } j = 0, \\ dp[i-1][j-1] + 1, & X_i = Y_j, \\ \max(dp[i-1][j], dp[i][j-1]), & X_i \neq Y_j. \end{cases}$$

Złożoność czasu i pamięci wynosi $\Theta(mn)$, choć pamięć dla samej długości można zredukować do $\Theta(\min(m, n))$.

2.4.5 Przykład: plecak 0/1

Dla przedmiotów o wagach w_i i wartościach v_i oraz pojemności W :

$dp[i][c]$ = maksymalna wartość z pierwszych i przedmiotów przy pojemności c .

Rekurencja:

$$dp[i][c] = \begin{cases} dp[i-1][c], & w_i > c, \\ \max(dp[i-1][c], dp[i-1][c-w_i] + v_i), & w_i \leq c. \end{cases}$$

Czas $\Theta(nW)$ jest pseudowielomianowy, bo zależy od wartości liczbowej W , a nie od liczby bitów jej zapisu.

2.4.6 Przykład: Floyd-Warshall

Dla najkrótszych ścieżek między wszystkimi parami:

$$d_{ij}^{(k)} = \min \left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \right).$$

Stan oznacza najkrótszą ścieżkę z i do j używającą jako pośrednich tylko wierzchołków z $\{1, \dots, k\}$. Czas $\Theta(n^3)$, pamięć $\Theta(n^2)$.

2.4.7 Przykład: Held-Karp dla TSP

Stan:

$dp[S][j]$ = koszt najkrótszej ścieżki startującej w 1, odwiedzającej S , kończącej w j .

Rekurencja:

$$dp[S][j] = \min_{i \in S, i \neq j} dp[S \setminus \{j\}][i] + c_{ij}.$$

Czas $\Theta(n^2 2^n)$ jest dużo lepszy niż brute force, ale nadal wykładniczy.

2.4.8 Pułapki

Najczęstsze błędy:

- stan nie zawiera całej informacji potrzebnej do przyszłych decyzji,
- rekurencja używa stanu, który nie został jeszcze obliczony,
- pomylenie indeksów i przypadków bazowych,
- nieświadome użycie pseudowielomianowej złożoności,
- odtworzenie rozwiązania nie jest zaplanowane,
- pamięć jest zbyt duża mimo akceptowalnego czasu.

2.4.9 Co powiedzieć na obronie

Programowanie dynamiczne stosuje się, gdy problem ma optymalną podstrukturę i nakładające się podproblemy. Definiujemy stan, rekurencję, przypadki bazowe i kolejność obliczeń. DP może być top-down z memoizacją albo bottom-up. Typowe przykłady to LCS, plecak, Floyd-Warshall, edycja napisów i Held-Karp dla TSP.

3 Zarządzanie przedsiębiorstwami informatycznymi

3.1 Czym charakteryzuje się projekt informatyczny?

Projekt informatyczny jest tymczasowym przedsięwzięciem prowadzącym do wytworzenia unikalnego produktu, usługi albo rezultatu związanego z systemem informatycznym. Może to być stworzenie aplikacji, wdrożenie systemu ERP, migracja danych, integracja usług, modernizacja infrastruktury, budowa hurtowni danych albo wdrożenie modelu uczenia maszynowego.

3.1.1 Cechy projektu

Projekt ma:

- jasno określony cel biznesowy albo techniczny,
- początek i koniec,
- ograniczony budżet, czas i zakres,
- interesariuszy,
- zespół projektowy,
- ryzyka i niepewność,
- plan realizacji,
- kryteria akceptacji.

W IT szczególnie istotne są niepewność wymagań, zmienność technologii, integracje z istniejącymi systemami, zależność od kompetencji ludzi oraz trudność szacowania prac twórczych.

3.1.2 Potrójne ograniczenie i jego rozszerzenia

Klasycznie projekt opisuje się przez trójkąt:

zakres — czas — koszt.

Zmiana jednego elementu wpływa na pozostałe. Jeśli zakres rośnie przy stałym terminie, trzeba zwiększyć zasoby, obniżyć jakość albo zaakceptować ryzyko opóźnienia.

W praktyce dodaje się jakość, ryzyko, satysfakcję klienta, wartość biznesową i ograniczenia organizacyjne. Projekt IT nie jest sukcesem tylko dlatego, że kod działa. Sukces oznacza dostarczenie wartości w akceptowalnym koszcie, czasie i jakości.

3.1.3 Specyfika projektów informatycznych

Projekty informatyczne mają kilka cech odróżniających je od wielu projektów inżynierii materialnej:

Niematerialny produkt Oprogramowania nie widać bezpośrednio, dlatego postęp bywa trudny do oceny, jeśli nie ma działających przyrostów.

Zmienność wymagań Użytkownicy często doprecyzowują potrzeby dopiero po zobaczeniu prototypu.

Wysoka złożoność zależności Integracje, dane, infrastruktura, bezpieczeństwo i wydajność tworzą zależności ukryte.

Niski koszt kopiowania, wysoki koszt wytworzenia Samo powielenie programu jest tanie, ale jego zaprojektowanie, testowanie i utrzymanie jest kosztowne.

Dług techniczny Decyzje przyspieszające krótkoterminowo mogą zwiększać koszt dalszego rozwoju.

Trudność szacowania Zadania programistyczne są często poznawczo nowe, więc estymacje mają dużą niepewność.

3.1.4 Interesariusze

Interesariusz to osoba albo organizacja wpływająca na projekt albo dotknięta jego wynikiem. W projekcie IT są to zwykle:

- sponsor biznesowy,
- product owner albo właściciel produktu,
- użytkownicy końcowi,
- zespół projektowy,
- administratorzy i zespoły utrzymaniowe,
- dział bezpieczeństwa,
- compliance i prawnicy,
- dostawcy zewnętrzni,
- klienci.

Dobre zarządzanie projektem wymaga rozumienia, kto podejmuje decyzje, kto akceptuje rezultat, kto dostarcza wymagania i kto ponosi skutki wdrożenia.

3.1.5 Zakres, wymagania i akceptacja

Zakres określa, co projekt ma dostarczyć. Wymagania mogą być:

- funkcjonalne - co system ma robić,
- niefunkcjonalne - wydajność, bezpieczeństwo, dostępność, użyteczność,
- biznesowe - jaki problem organizacji ma zostać rozwiązany,
- techniczne - standardy, technologie, integracje,
- regulacyjne - zgodność z prawem i politykami.

Kryteria akceptacji muszą być weryfikowalne. Zamiast "system ma być szybki" lepiej zapisać "95 percent odpowiedzi API dla operacji X ma czas poniżej 300 ms przy obciążeniu Y".

3.1.6 Ryzyka typowe dla projektów IT

- niedoszacowanie złożoności,
- niestabilne wymagania,
- brak dostępności kluczowych osób,
- opóźnienia integracji,
- niska jakość danych,
- problemy z bezpieczeństwem,
- vendor lock-in,
- brak środowisk testowych,
- niedostateczne testy,
- brak planu migracji i rollbacku,
- konflikt między terminem a jakością.

3.1.7 Co powiedzieć na obronie

Projekt informatyczny to tymczasowe przedsięwzięcie tworzące unikalny rezultat IT. Charakteryzuje się określonym celem, zakresem, terminem, budżetem, interesariuszami i ryzykiem. Szczególne dla IT są zmienne wymagania, niematerialność produktu, trudność szacowania, złożoność integracji i długi techniczny.

3.2 Główne różnice między projektem a pracą operacyjną

Projekt i praca operacyjna są potrzebne organizacji, ale mają różny charakter. Projekt wprowadza zmianę; operacje utrzymują ciągłe działanie.

3.2.1 Porównanie

Kryterium	Projekt	Praca operacyjna
-----------	---------	------------------

Czas trwania	tymczasowy, ma początek i koniec	ciągła, powtarzalna
Cel	wytworzenie unikalnego rezultatu albo zmiany	utrzymanie działania i świadczenie usług
Zakres	zdefiniowany dla konkretnego przedsięwzięcia	stabilny, procesowy
Niepewność	zwykle wysoka	zwykle niższa, procedury są znane
Sukces	dostarczenie wartości w czasie, budżecie i jakości	stabilność, efektywność, SLA, jakość obsługi
Zespół	często czasowy i interdyscyplinarny	stały albo rotacyjny
Przykład IT	budowa nowej aplikacji	utrzymanie aplikacji produkcyjnej

3.2.2 Relacja między projektem i operacjami

Projekt często przekazuje wynik do operacji. Przykład: zespół projektowy wdraża system, a potem zespół utrzymaniowy monitoruje go, obsługuje incydenty, wykonuje aktualizacje i wspiera użytkowników.

Granica nie zawsze jest ostra. Rozwój produktu cyfrowego może być ciągłym strumieniem zmian, ale poszczególne większe inicjatywy nadal mają charakter projektowy. W podejściu produktowym mówi się częściej o ciągłym rozwoju produktu niż o jednorazowych projektach, jednak nadal występują ograniczone inicjatywy, budżety i cele.

3.2.3 Przykład

Wdrożenie modułu płatności online jest projektem: ma zakres, termin, integracje, testy bezpieczeństwa i odbiór. Codzienne monitorowanie transakcji, reagowanie na błędy płatności i aktualizacja certyfikatów to operacje.

3.2.4 Co powiedzieć na obronie

Projekt jest tymczasowy i tworzy unikalną zmianę, a praca operacyjna jest ciągła i powtarzalna. Projekt kończy się przekazaniem rezultatu, natomiast operacje utrzymują usługę, proces albo system. W IT projekty często budują lub zmieniają systemy, a operacje zapewniają ich stabilne działanie.

3.3 Metodyki zarządzania projektami: charakterystyka, różnice i cechy wspólne

Metodyka zarządzania projektem określa sposób planowania, organizacji, realizacji, kontroli i zamykania prac. W IT najczęściej porównuje się podejścia predykcyjne, zwinne i hybrydowe.

3.3.1 Waterfall

Waterfall, czyli model kaskadowy, zakłada sekwencyjne fazy:

1. analiza wymagań,
2. projekt,
3. implementacja,
4. testowanie,
5. wdrożenie,

6. utrzymanie.

Zalety:

- jasny plan i dokumentacja,
- dobre dopasowanie do stałych wymagań,
- łatwiejsze kontraktowanie zakresu,
- formalna kontrola zmian.

Wady:

- późna informacja zwrotna od użytkownika,
- kosztowne zmiany wymagań,
- ryzyko odkrycia problemów dopiero pod koniec,
- słabe dopasowanie do niepewnych produktów cyfrowych.

Waterfall jest sensowny, gdy wymagania są stabilne, środowisko regulowane, a koszt zmian wysoki, np. wybrane systemy krytyczne albo kontrakty publiczne.

3.3.2 PMBOK i podejście procesowe

PMBOK nie jest jedną sekwencją prac, lecz zbiorem dobrych praktyk zarządzania projektami. Obejmuje obszary takie jak zakres, harmonogram, koszt, jakość, zasoby, komunikacja, ryzyko, zamówienia i interesariusze. Jest metodycznie neutralny: można go stosować w projektach kaskadowych, zwinnych i hybrydowych.

Mocną stroną PMBOK jest kompletność zarządcza. Słabością w małych zespołach może być nadmiar formalizacji, jeśli stosuje się go bez dostosowania.

3.3.3 PRINCE2

PRINCE2 jest metodyką silnie nastawioną na uzasadnienie biznesowe, role, produkty zarządcze i kontrolę etapową. Kluczowe elementy:

- ciągła zasadność biznesowa,
- uczenie się z doświadczeń,
- zdefiniowane role i odpowiedzialności,
- zarządzanie etapami,
- zarządzanie przez wyjątki,
- koncentracja na produktach,
- dostosowanie do środowiska projektu.

PRINCE2 dobrze pasuje do organizacji wymagających kontroli i raportowania, ale może być zbyt formalny dla małych, szybko zmieniających się produktów.

3.3.4 Agile

Agile to rodzina podejść opartych na iteracyjności, przyrostowym dostarczaniu wartości, współpracy z klientem i reagowaniu na zmianę. Nie oznacza braku planu. Oznacza planowanie adaptacyjne i częstą

weryfikację założeń.

Wspólne idee:

- krótkie iteracje,
- częste dostarczanie działającego przyrostu,
- bliska współpraca z użytkownikiem lub właścicielem produktu,
- priorytetyzowany backlog,
- retrospekcje i ciągłe usprawnianie,
- akceptacja zmienności wymagań.

3.3.5 Scrum

Scrum jest ramą pracy zwinnej. Role:

Product Owner odpowiada za wartość produktu i backlog.

Scrum Master dba o proces, usuwa przeszkody i wspiera zespół.

Developers wytwarzają przyrost produktu.

Zdarzenia:

- Sprint,
- Sprint Planning,
- Daily Scrum,
- Sprint Review,
- Sprint Retrospective.

Artefakty:

- Product Backlog,
- Sprint Backlog,
- Increment.

Scrum dobrze działa, gdy produkt można rozwijać przyrostowo, a wymagania są zmienne. Nie rozwiązuje sam problemów technicznych: wymaga praktyk inżynierskich, testów, CI/CD i dbania o architekturę.

3.3.6 Kanban

Kanban koncentruje się na przepływie pracy. Typowe zasady:

- wizualizacja pracy na tablicy,
- limity WIP, czyli ograniczenie pracy w toku,
- zarządzanie przepływem,
- jawne polityki procesu,
- ciągłe doskonalenie.

Kanban jest dobry dla utrzymania, zespołów operacyjnych, obsługi zgłoszeń i środowisk, gdzie praca napływa nieregularnie. W przeciwieństwie do Scruma nie wymaga sprintów.

3.3.7 Extreme Programming

XP jest metodyką zwinną silnie skupioną na praktykach inżynierskich:

- pair programming,
- test-driven development,
- continuous integration,
- refaktoryzacja,
- prosta architektura,
- małe wydania,
- wspólna własność kodu.

XP odpowiada na lukę często widoczną w Scrumie: sama organizacja iteracji nie gwarantuje jakości technicznej.

3.3.8 Lean

Lean skupia się na maksymalizacji wartości i eliminacji marnotrawstwa. W IT marnotrawstwem może być nadmiar funkcji, oczekiwanie, przełączanie kontekstu, częściowo wykonana praca, defekty, ręczne czynności i nadmierna biurokracja. Lean podkreśla optymalizację całego strumienia wartości, nie tylko lokalną produktywność osób.

3.3.9 DevOps

DevOps nie jest klasyczną metodyką projektową, ale zestawem praktyk łączących rozwój, utrzymanie i automatyzację dostarczania. Ważne elementy:

- CI/CD,
- infrastructure as code,
- monitoring i observability,
- automatyzacja testów,
- szybki rollback,
- współodpowiedzialność za produkcję.

DevOps skraca dystans między projektem i operacjami.

3.3.10 Podejścia hybrydowe

W praktyce często łączy się elementy. Przykład: formalne etapy i budżet według PRINCE2, a realizacja zespołu w Scrumie. Albo roadmapa produktu i kwartalne cele, ale codzienny przepływ w Kanbanie. Hybryda jest sensowna, jeśli wynika z potrzeb organizacji, a nie z niekonsekwencji.

3.3.11 Cechy wspólne metodyk

Mimo różnic większość metodyk obejmuje:

- definiowanie celu i zakresu,

- role i odpowiedzialności,
- planowanie pracy,
- monitorowanie postępu,
- zarządzanie ryzykiem,
- komunikację z interesariuszami,
- kontrolę jakości,
- obsługę zmian,
- zamknięcie albo przekazanie rezultatu.

3.3.12 Najważniejsze różnice

Waterfall zakłada stabilność wymagań i sekwencyjność. Agile zakłada zmienność i iteracyjne dostarczanie wartości. Scrum daje ramę pracy zespołowej w iteracjach. Kanban optymalizuje przepływ i limity pracy w toku. PRINCE2 i PMBOK są bardziej zarządcze i formalne. XP skupia się na jakości inżynierskiej. DevOps skupia się na automatyzacji dostarczania i odpowiedzialności za działanie systemu.

3.3.13 Co powiedzieć na obronie

Metodyki różnią się podejściem do zmiany, planowania i kontroli. Kaskadowe są dobre przy stabilnych wymaganiach, zwinne przy niepewności i potrzebie częstej informacji zwrotnej, Kanban przy ciągłym przepływie zadań, PRINCE2/PMBOK przy formalnym zarządzaniu, XP przy nacisku na praktyki programistyczne. Wspólne są cel, role, planowanie, monitorowanie, ryzyko, jakość i komunikacja.

4 Reprezentacja wiedzy

4.1 Metoda rezolucji w rachunku predykatów

Rezolucja jest regułą wnioskowania i zarazem podstawą automatycznego dowodzenia twierdzeń. Jej główna idea polega na sprowadzeniu dowodu $T \models \beta$ do wykazania sprzeczności zbioru $T \cup \{\neg\beta\}$. Jeśli założenia razem z negacją tezy są niespełnialne, to teza wynika z założeń.

4.1.1 Idea dowodu przez sprzeczność

Dla teorii T i zdania β zachodzi:

$$T \models \beta \iff T \cup \{\neg\beta\} \text{ jest niespełnialny.}$$

Rezolucja próbuje więc wyprowadzić klauzulę pustą, oznaczaną często $\square$. Klauzula pusta reprezentuje fałsz, czyli sprzeczność.

W rachunku zdań metoda jest decyzyjna: jeśli zbiór klauzul jest niespełnialny, rezolucja może znaleźć sprzeczność; jeśli jest spełnialny, pełne przeszukanie może to wykazać. W rachunku predykatów pierwszego rzędu sytuacja jest trudniejsza: logika pierwszego rzędu jest półrozstrzygalna. Jeśli wniosek rzeczywiście wynika, procedura może znaleźć dowód; jeśli nie wynika, przeszukiwanie może nie zakończyć się.

4.1.2 Literały i klauzule

Atom to formuła postaci $P(t_1, \dots, t_k)$, gdzie P jest predykatem, a t_i są termami. Literał to atom albo negacja atomu. Klauzula jest alternatywą literałów:

$$L_1 \vee L_2 \vee \dots \vee L_m.$$

Zbiór klauzul interpretuje się jako koniunkcję wszystkich klauzul. Klauzula pusta $\square$ jest alternatywą zera literałów, czyli jest zawsze fałszywa.

4.1.3 Reguła rezolucji w rachunku zdań

Jeśli mamy dwie klauzule zawierające literały komplementarne:

$$\lambda \vee A, \quad \neg \lambda \vee B,$$

to możemy wyprowadzić rezolwentę:

$$A \vee B.$$

Reguła jest poprawna semantycznie, bo jeśli pierwsza i druga klauzula są prawdziwe, to po usunięciu pary komplementarnej pozostała alternatywa musi być prawdziwa.

Przykład:

$$P \vee Q, \quad \neg P \vee R$$

dają:

$$Q \vee R.$$

4.1.4 Sprowadzenie formuł do postaci klauzulowej

Aby stosować rezolucję, formuły trzeba zamienić na postać normalną. Dla rachunku zdań używa się CNF:

$$\bigwedge_i \bigvee_j L_{ij}.$$

Algorytm:

1. usunąć implikacje i równoważności:

$$\alpha \rightarrow \beta \equiv \neg \alpha \vee \beta,$$

$$\alpha \leftrightarrow \beta \equiv (\neg \alpha \vee \beta) \wedge (\neg \beta \vee \alpha);$$

2. przesunąć negacje do atomów, używając praw de Morgana;
3. zastosować rozdzielność $\vee$ względem $\wedge$;
4. potraktować koniunkcję jako zbiór klauzul.

4.1.5 Dodatkowe problemy w rachunku predykatów

W rachunku predykatów pojawiają się kwantyfikatory i zmienne. Dwa literały mogą nie być identyczne syntaktycznie, ale mogą stać się identyczne po podstawieniu. Przykład:

$$Student(x), \quad Student(Piotr).$$

Nie są tym samym literałem, ale podstawienie $\{x/Piotr\}$ je uzgadnia. Do rezolucji pierwszego rzędu potrzebujemy więc:

- postaci preneksowej i klauzulowej,
- standaryzacji nazw zmiennych,
- skolemizacji,
- unifikacji,
- reguły rezolucji z najogólniejszym unifikatorem.

4.1.6 Postać preneksowa

Formuła jest w postaci preneksowej, gdy wszystkie kwantyfikatory stoją z przodu:

$$Q_1x_1Q_2x_2\ldots Q_nx_n. M,$$

gdzie M jest formułą bez kwantyfikatorów, nazywaną macierzą.

Przekształcanie do postaci preneksowej obejmuje:

1. usunięcie implikacji i równoważności,
2. przesunięcie negacji do atomów:

$$\neg\forall x \alpha \equiv \exists x \neg\alpha, \quad \neg\exists x \alpha \equiv \forall x \neg\alpha,$$

3. zmianę nazw zmiennych związanych, aby uniknąć kolizji,
4. wyniesienie kwantyfikatorów na początek,
5. przekształcenie macierzy do CNF.

4.1.7 Skolemizacja

Skolemizacja usuwa kwantyfikatory egzystencjalne, zastępując zmienne egzystencjalne stałymi albo funkcjami Skolema. Nie zachowuje równoważności logicznej wprost, ale zachowuje spełnialność, co wystarcza dla rezolucji refutacyjnej.

Jeśli mamy:

$$\exists y P(y),$$

zastępujemy y nową stałą c :

$$P(c).$$

Jeśli zmienna egzystencjalna zależy od wcześniejszych uniwersalnych:

$$\forall x \exists y Loves(x, y),$$

to y zastępujemy funkcją Skolema $f(x)$:

$$\forall x Loves(x, f(x)).$$

Po skolemizacji wszystkie pozostałe zmienne są traktowane jako uniwersalnie kwantyfikowane, więc kwantyfikatory uniwersalne można pominąć w zapisie klauzul.

4.1.8 Unifikacja

Unifikacja znajduje podstawienie, które czyni dwa termy albo literały identycznymi. Najważniejszy jest najogólniejszy unifikator, MGU, czyli taki unifikator, z którego każdy inny unifikator może być otrzymany przez dalsze podstawienie.

Przykłady:

$$P(x, a) \quad \text{i} \quad P(f(y), a)$$

mają MGU:

$$\{x/f(y)\}.$$

Literały:

$$\text{Parent}(x, \text{John}), \quad \text{Parent}(\text{Mary}, y)$$

mają MGU:

$$\{x/\text{Mary}, y/\text{John}\}.$$

W unifikacji trzeba stosować occurs check: zmienna nie może zostać podstawiona termem zawierającym tę samą zmienną, np. $x = f(x)$ nie powinno być dopuszczone w standardowej logice pierwszego rzędu, bo tworzyłoby nieskończony term.

4.1.9 Reguła rezolucji pierwszego rzędu

Jeśli mamy klauzule:

$$L \vee A, \quad \neg K \vee B,$$

a θ jest MGU literałów L i K , to rezolwentą jest:

$$(A \vee B)\theta.$$

Najpierw unifikujemy literały komplementarne, a potem usuwamy je i stosujemy podstawienie do reszty klauzuli.

Przykład:

$$\text{Human}(x) \rightarrow \text{Mortal}(x), \quad \text{Human}(\text{Socrates}).$$

Po CNF:

$$\neg \text{Human}(x) \vee \text{Mortal}(x), \quad \text{Human}(\text{Socrates}).$$

Unifikujemy $\text{Human}(x)$ z $\text{Human}(\text{Socrates})$ przez $\{x/\text{Socrates}\}$ i otrzymujemy:

$$\text{Mortal}(\text{Socrates}).$$

Jeśli chcemy udowodnić $\text{Mortal}(\text{Socrates})$ przez refutację, dodajemy $\neg \text{Mortal}(\text{Socrates})$, a potem rezolucja daje klauzulę pustą.

4.1.10 Procedura dowodzenia rezolucyjnego

1. Weź teorię T i tezę β .
2. Dodaj negację tezy: $T \cup \{\neg\beta\}$.
3. Przekształć wszystkie formuły do postaci klauzulowej: usunięcie implikacji, przesunięcie negacji, standaryzacja zmiennych, preneks, skolemizacja, CNF, usunięcie kwantyfikatorów uniwersalnych.
4. Wybieraj pary klauzul z unifikowalnymi literałami komplementarnymi.
5. Dodawaj rezolwentę do zbioru klauzul.
6. Jeśli pojawi się $\square$, dowód zakończony sukcesem.

4.1.11 Poprawność, pełność i problem strategii

Rezolucja jest poprawna: każda wyprowadzona klauzula wynika logicznie z poprzednich. Jest też pełna refutacyjnie dla logiki pierwszego rzędu: jeśli zbiór klauzul jest niespełnialny, istnieje dowód rezolucyjny prowadzący do klauzuli pustej.

Pełność nie oznacza efektywności. Naiwne generowanie wszystkich rezolwent prowadzi do eksplozji kombinatorycznej. Dlatego stosuje się strategię:

- rezolucję liniową,
- set of support,
- preferencję klauzul jednostkowych,
- subsumpcję,
- porządkowanie literałów,
- indeksowanie termów,
- ograniczenia głębokości albo heurystyki wyboru.

4.1.12 Rezolucja SLD i Prolog

W programowaniu logicznym, szczególnie w Prologu, używa się wyspecjalizowanej rezolucji SLD dla klauzul Horna. Klauzula Horna ma co najwyżej jeden literał pozytywny. Reguła:

$$Head \leftarrow Body_1, \dots, Body_k$$

odpowiada formule:

$$Body_1 \wedge \dots \wedge Body_k \rightarrow Head.$$

Prolog rozwiązuje zapytanie przez próbę refutacji jego negacji, wybierając cele od lewej do prawej i klauzule w kolejności programu. To jest efektywne i praktyczne, ale strategia Prologa nie jest pełna dla wszystkich możliwych programów z powodu deterministycznego porządku przeszukiwania i możliwości zapętlenia.

4.1.13 Co powiedzieć na obronie

Rezolucja w rachunku predykatów polega na dowodzeniu przez sprzeczność: do teorii dodaje się negację tezy, przekształca wszystko do zbioru klauzul, usuwa kwantyfikatory egzystencjalne przez skolemizację, a literały dopasowuje przez unifikację. Reguła rezolucji usuwa parę literałów komplementarnych po zastosowaniu MGU. Wyprowadzenie klauzuli pustej oznacza, że teza wynika z teorii. Metoda jest poprawna i pełna refutacyjnie, ale wymaga strategii, bo liczba rezolwent może eksplodować.

4.2 Główne problemy w zagadnieniach wnioskowania o działaniach

Wnioskowanie o działaniach dotyczy systemów dynamicznych: mamy obiekty, ich własności oraz akcje zmieniające stan świata. Celem jest formalne odpowiadanie na pytania typu: co będzie prawdziwe po wykonaniu akcji, czy akcja jest wykonalna, czy jakiś cel da się osiągnąć sekwencją akcji, jakie skutki są konieczne, a jakie możliwe.

4.2.1 Model dynamicznego świata

Stan świata opisuje wartości fluentów, czyli własności mogących zmieniać się w czasie. Akcje przekształcają stany. W najprostszym modelu mamy:

- zbiór fluentów F ,
- zbiór akcji Ac ,
- stany jako interpretacje fluentów,
- stan początkowy,
- funkcję przejścia $Res(A, \sigma)$ zwracającą możliwe stany po wykonaniu akcji A w stanie σ .

Jeśli akcja jest deterministyczna, $Res(A, \sigma)$ ma jeden element. Jeśli niedeterministyczna, może mieć wiele wyników. Jeśli akcja jest niewykonalna, wynik może być pusty albo zgodnie z przyjętą semantyką może oznaczać brak efektu.

4.2.2 Ontologia działań

W modelowaniu działań trzeba zdecydować, jakie typy zjawisk dopuszczamy:

- skutki deterministyczne albo niedeterministyczne,
- skutki pewne albo typowe, czyli domyślne,
- skutki bezpośrednie i pośrednie,
- skutki natychmiastowe i opóźnione,
- akcje jednoetapowe albo trwające w czasie,
- akcje sekwencyjne albo współbieżne,
- akcje elementarne albo złożone,
- pełną albo niepełną wiedzę o stanie i akcjach.

Każda decyzja upraszcza albo komplikuje semantykę. Prosty język akcji może wystarczyć dla sekwencyjnych działań deterministycznych, ale zawiedzie przy skutkach ubocznych, współbieżności albo wyjątkach.

4.2.3 Frame problem

Frame problem to problem reprezentowania i wnioskowania o tym, co nie zmienia się po wykonaniu akcji. W świecie z wieloma fluentami większość własności zwykle pozostaje bez zmian. Naiwne dopisywanie dla każdej akcji i każdego fluentu aksjomatu "ta akcja nie zmienia tej własności" prowadzi do ogromnej liczby aksjomatów.

Przykład: w Yale Shooting Problem załadowanie broni zmienia fluent *loaded*, ale nie powinno zmieniać faktu, że Fred żyje. Logika klasyczna bez dodatkowej zasady inercji nie pozwala automatycznie wywnioskować, że Fred nadal żyje po załadowaniu broni.

Rozwiązaniem jest zasada inercji: fluent zachowuje wartość, dopóki akcja nie wymusza zmiany albo nie zwalnia fluentu spod inercji. Formalizacje akcji często budują tę zasadę w semantykę, zamiast wypisywać wszystkie frame axioms.

4.2.4 Ramification problem

Ramification problem dotyczy skutków pośrednich. Akcja może bezpośrednio zmieniać jeden fluent, ale przez ograniczenia stanu wymuszać zmiany innych fluentów.

Przykład: jeśli obowiązuje reguła $walking \rightarrow alive$, a strzał powoduje $\neg alive$, to pośrednio powinno wynikać $\neg walking$. Nie chcemy ręcznie wypisywać każdego skutku pośredniego każdej akcji.

Trudność polega na tym, aby:

- reprezentować ograniczenia statyczne,
- propagować skutki bezpośrednio przez te ograniczenia,
- nie generować nieintuicyjnych skutków,
- odróżnić skutki pośrednie od warunków wykonalności.

W slajdach pojawia się przykład, w którym próba zachowania spójności ograniczeń może prowadzić do dziwnych wniosków, np. "ożywienia" martwego indyka przy akcji zachęcania do chodzenia. To pokazuje, że sama minimalizacja zmian bywa niewystarczająca.

4.2.5 Qualification problem

Qualification problem dotyczy warunków, pod którymi akcja jest możliwa i przynosi oczekiwane skutki. W praktyce liczba potencjalnych warunków jest ogromna i nie da się ich wszystkich wypisać.

Klasyczny przykład: aby uruchomić samochód, trzeba mieć kluczyk, paliwo, sprawny akumulator, sprawne przewody i bardzo wiele innych rzeczy, w tym brak ziemniaka w rurze wydechowej. Nie da się w każdym modelu jawnie sprawdzać wszystkich nietypowych przeszkód.

Rozwiązaniem mogą być:

- domyślne założenia normalności,
- logiki niemonotoniczne,
- preconditions z wyjątkami,
- probabilistyczne modele skuteczności działań,
- rozróżnienie skutków typowych i nietypowych.

4.2.6 Problem skutków domyślnych

W realnym świecie działania mają często skutki typowe, ale niepewne. Strzał z załadowanej broni typowo zabija, ale mogą istnieć wyjątki: broń się zacina, cel ma osłonę, strzał chybia. Logika klasyczna jest monotoniczna: dodanie nowej informacji nie usuwa wcześniejszych wniosków. Rozumowanie domyślne jest niemonotoniczne: wniosek "typowo P " może zostać wycofany po poznaniu wyjątku.

Modele działań z efektami domyślnymi muszą odróżniać:

- skutki pewne,
- skutki normalne albo preferowane,
- skutki atypowe,
- minimalizację odstępstw od normalności.

4.2.7 Niedeterminizm

Akcja może mieć wiele możliwych rezultatów. Przykład: rzut kostką, próba otwarcia uszkodzonych drzwi, ruch robota po śliskiej powierzchni. Wtedy pytania dzielą się na:

- czy cel jest konieczny po akcji, czyli zachodzi we wszystkich możliwych wynikach,
- czy cel jest możliwy, czyli zachodzi w co najmniej jednym wyniku,
- czy istnieje plan gwarantujący cel,
- czy istnieje strategia reagująca na obserwacje.

Niedeterminizm zwiększa złożoność planowania, bo pojedyncza sekwencja akcji może nie wystarczyć; potrzebna może być strategia warunkowa.

4.2.8 Współbieżność

Wiele akcji może zachodzić jednocześnie. Pojawiają się pytania:

- kiedy akcje są konfliktowe,
- czy skutki akcji złożonej są sumą skutków akcji elementarnych,
- czy jedna akcja blokuje drugą,
- czy akcje razem mają skutek inny niż osobno.

Przykład: dwie niezależne akcje, jak otwarcie drzwi i podniesienie miski, mogą mieć łączny skutek będący koniunkcją skutków. Ale dwie akcje podnoszenia miski jedną ręką mogą osobno rozlać zupę, a razem już nie. To łamie prostą zasadę dziedziczenia skutków.

4.2.9 Czas, akcje trwające i scenariusze

Jeśli akcje trwają w czasie, trzeba reprezentować:

- moment rozpoczęcia i zakończenia,
- wartości fluentów w trakcie akcji,
- skutki opóźnione,
- akcje wyzwalane przez stany,
- akcje wyzwalane przez inne akcje,
- scenariusze jako zbiory wystąpień akcji w czasie.

W modelach z czasem liniowym stan w chwili $t + 1$ zależy od stanu w chwili t , wykonanych akcji i ewentualnych reguł dynamicznych. W modelach z czasem rozgałęzionym analizuje się wiele możliwych przyszłości.

4.2.10 Niepełna wiedza i obserwacje

Agent często nie zna pełnego stanu świata. Wtedy zamiast pojedynczego stanu ma zbiór stanów możliwych albo przekonanie probabilistyczne. Akcje mogą być:

- ontic, czyli zmieniać świat,

- sensing, czyli dostarczać informacji,
- announcement, czyli zmieniać wiedzę agentów.

Wnioskowanie musi wtedy rozróżniać prawdę w świecie od wiedzy albo przekonań agenta. Stąd powiązanie z logikami epistemicznymi, modalnymi i dynamicznymi.

4.2.11 Planowanie

Wnioskowanie o działaniach jest ściśle związane z planowaniem. Typowe zapytania:

- Czy po sekwencji akcji $\langle A_1, \dots, A_n \rangle$ zachodzi α ?
- Czy istnieje sekwencja akcji prowadząca do celu?
- Czy każda realizacja akcji prowadzi do celu?
- Czy akcja jest wykonalna w danym stanie?
- Jakie fluenty są obserwowalne po akcji?

W modelu niedeterministycznym albo niepełnoinformacyjnym plan może być warunkowy: następna akcja zależy od obserwacji.

4.2.12 Co powiedzieć na obronie

Główne problemy wnioskowania o działaniach to frame problem, ramification problem i qualification problem. Frame problem dotyczy tego, co pozostaje bez zmian; ramification problem skutków pośrednich; qualification problem ogromnej liczby warunków potrzebnych do skutecznego wykonania akcji. Dodatkowo ważne są niedeterminizm, skutki domyślne, współbieżność, czas trwania akcji, niepełna wiedza i planowanie. Formalne języki akcji próbują tak dobrać semantykę, aby uzyskać intuicyjne wnioski bez ręcznego wypisywania wszystkich wyjątków i skutków.

5 Metody sztucznej inteligencji i systemy ekspertowe

5.1 Sposób działania algorytmu ewolucyjnego

Algorytm ewolucyjny jest populacyjną metaheurystyką inspirowaną ewolucją biologiczną. Przetwarza zbiór kandydatów na rozwiązania, stosuje selekcję, rekombinację i mutację, a jakość kandydatów ocenia funkcją dopasowania. Celem jest iteracyjne przesuwanie populacji w kierunku lepszych rozwiązań przy zachowaniu wystarczającej różnorodności.

5.1.1 Elementy algorytmu

Osobnik kandydat na rozwiązanie problemu.

Genotyp reprezentacja osobnika używana przez operatory, np. bitstring, wektor liczb rzeczywistych, permutacja.

Fenotyp rozwiązanie po dekodowaniu genotypu, np. konkretna trasa TSP.

Populacja zbiór osobników.

Funkcja dopasowania miara jakości osobnika. Dla problemu minimalizacji często przekształca się koszt na dopasowanie albo stosuje selekcję opartą bezpośrednio na rankingu.

Selekcja wybór osobników do reprodukcji.

Krzyżowanie tworzenie potomków z informacji wielu rodziców.

Mutacja losowa modyfikacja osobnika.

Elityzm przenoszenie najlepszych osobników bez zmian do kolejnego pokolenia.

5.1.2 Ogólny schemat

1. Zainicjalizuj populację, losowo albo heurystycznie.
2. Oceń każdego osobnika funkcją dopasowania.
3. Dopóki nie spełniono warunku stopu:
 - (a) wybierz rodziców,
 - (b) zastosuj krzyżowanie,
 - (c) zastosuj mutację,
 - (d) oceń potomków,
 - (e) utwórz nowe pokolenie,
 - (f) zachowaj elity, jeśli stosujesz elityzm.
4. Zwróć najlepszego znalezionej osobnika.

Warunek stopu może zależeć od liczby pokoleń, budżetu czasu, liczby ewaluacji funkcji celu, braku poprawy albo osiągnięcia wymaganej jakości.

5.1.3 Reprezentacja

Reprezentacja jest kluczowa. Powinna:

- kodować wszystkie istotne rozwiązania,
- ograniczać liczbę rozwiązań niedopuszczalnych,
- pozwalać operatorom tworzyć sensownych potomków,
- zachowywać strukturę problemu.

Przykłady:

- bitstring dla wyboru podzbioru cech,
- wektor rzeczywisty dla optymalizacji ciągłej,
- permutacja dla TSP i harmonogramowania,
- drzewo dla programowania genetycznego.

Jeśli reprezentacja generuje rozwiązania niedopuszczalne, trzeba stosować kary, naprawę, dekodery albo specjalne operatory zachowujące dopuszczalność.

5.1.4 Selekcja

Selekcja zwiększa prawdopodobieństwo reprodukcji lepszych osobników, ale nie powinna zbyt szybko usuwać różnorodności.

Ruletka prawdopodobieństwo wyboru proporcjonalne do dopasowania. Wrażliwa na skalowanie dopasowania.

Selekcja rankingowa osobniki sortuje się według jakości, a prawdopodobieństwa zależą od rangi. Jest stabilniejsza.

Turniej losuje się kilka osobników i wybiera najlepszego. Rozmiar turnieju kontroluje presję selekcyjną.

Elityzm najlepsze osobniki przechodzą do następnego pokolenia, co zapobiega utracie najlepszego znalezionego rozwiązania.

5.1.5 Krzyżowanie i mutacja

Dla bitstringów używa się krzyżowania jednopunktowego, wielopunktowego albo jednorodnego. Dla wektorów rzeczywistych można stosować średnie arytmetyczne, BLX- α , SBX albo rekombinację liniową. Dla permutacji potrzebne są operatory zachowujące permutację, np. PMX, OX, CX.

Mutacja zapewnia eksplorację. Przykłady:

- odwrócenie bitu,
- dodanie szumu Gaussowskiego,
- zamiana dwóch pozycji w permutacji,
- odwrócenie fragmentu trasy,
- mutacja poddrzewa w programowaniu genetycznym.

5.1.6 Eksploracja i eksploatacja

Algorytm ewolucyjny musi równoważyć:

Eksplorację przeszukiwanie nowych regionów przestrzeni.

Eksploatację intensywne ulepszanie obiecujących regionów.

Zbyt duża presja selekcyjna albo zbyt mała mutacja prowadzą do przedwczesnej zbieżności. Zbyt duża losowość powoduje zachowanie podobne do losowego przeszukiwania. Różnorodność można utrzymywać przez mutację, niching, crowding, fitness sharing, restart albo populacje wyspowe.

5.1.7 Parametry

Najważniejsze parametry:

- rozmiar populacji,
- prawdopodobieństwo krzyżowania,
- prawdopodobieństwo mutacji,
- siła mutacji,
- presja selekcyjna,
- liczba elit,
- kryterium stopu.

Parametry silnie wpływają na wynik, dlatego należy je stroić na walidacyjnych instancjach i raportować w eksperymentach.

5.1.8 Co powiedzieć na obronie

Algorytm ewolucyjny utrzymuje populację rozwiązań, ocenia je funkcją dopasowania, wybiera lepsze osobniki do reprodukcji i tworzy nowe rozwiązania przez krzyżowanie oraz mutację. Selekcja daje presję w stronę jakości, mutacja i różnorodność zapewniają eksplorację, a elityzm chroni najlepsze rozwiązania. Kluczowe są reprezentacja i dobór operatorów zgodnych z problemem.

5.2 Porównanie algorytmu ewolucyjnego z metodami optymalizacji w $\mathbb{R}^n$

Algorytmy ewolucyjne i klasyczne metody optymalizacji ciągłej rozwiązują podobny problem poszukiwania ekstremum, ale mają inne założenia, koszty i gwarancje.

5.2.1 Algorytm ewolucyjny

Algorytm ewolucyjny:

- jest populacyjny,
- zwykle nie wymaga gradientu,
- dobrze radzi sobie z funkcjami niegładkimi, zaszumionymi i wielomodalnymi,
- łatwo obsługuje reprezentacje dyskretne i mieszane,
- może znajdować dobre rozwiązania globalnie, ale bez gwarancji optimum,
- wymaga wielu ewaluacji funkcji celu.

Jakość wyniku jest losowa i zależy od parametrów oraz budżetu obliczeniowego. Wyniki powinno się raportować statystycznie.

5.2.2 Metoda największego spadku

Gradient descent:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

jest metodą lokalną, deterministyczną przy ustalonym punkcie startowym i kroku. Dla funkcji gładkich wypukłych ma gwarancje zbieżności, a dla silnie wypukłych zbieżność liniową. Wymaga gradientu i jest wrażliwa na skalowanie zmiennych oraz uwarunkowanie. W funkcjach niewypukłych może utknąć w minimum lokalnym albo punkcie siodłowym.

5.2.3 Metoda Nelder-Meada

Nelder-Mead jest bezpochodną metodą lokalną. Utrzymuje sympleks $n + 1$ punktów i wykonuje operacje odbicia, ekspansji, kontrakcji i redukcji. Dobrze działa w małych wymiarach i dla funkcji bez gradientu, ale w dużych wymiarach traci efektywność. Nie jest populacyjny w sensie ewolucyjnym, choć utrzymuje wiele punktów. Ma słabsze gwarancje zbieżności niż metody gradientowe w problemach gładkich wypukłych.

5.2.4 Metody quasi-Newtonowskie

BFGS i L-BFGS korzystają z gradientu i przybliżają krzywiznę funkcji. Dla gładkich problemów są zwykle znacznie szybsze niż gradient descent. W problemach wypukłych mogą dawać bardzo wysoką dokładność. Wymagają jednak gradientu lub jego dobrego przybliżenia, a dla funkcji silnie zaszumionych albo niedyskretnej reprezentacji mogą być nieodpowiednie.

5.2.5 Porównanie jakości i sposobu działania

Kryterium	Algorytm ewolucyjny	Metody wypukłe/lokalne
Informacja o funkcji	wartości funkcji, czasem ograniczenia	gradient, czasem Hessian albo pochodne
Charakter	globalna metaheurystyka populacyjna	lokalna metoda analityczna
Wypukłość	nie wymaga wypukłości	wypukłość daje silne gwarancje
Gładkość	nie wymaga gładkości	zwykle wymaga gładkości
Koszt	wiele ewaluacji funkcji celu	mniej iteracji, ale każda może wymagać gradientu/Hessianu
Wynik	losowy, zależny od budżetu	deterministyczny przy ustalonych parametrach
Gwarancje	zwykle słabe formalnie	silne dla problemów wypukłych
Duży wymiar	często trudny bez specjalizacji	L-BFGS/SGD skalują się dobrze

5.2.6 Kiedy wybrać którą metodę

Algorytm ewolucyjny warto wybrać, gdy:

- funkcja jest czarnoskrzynkowa,
- nie ma gradientu,
- przestrzeń jest dyskretna albo mieszana,
- problem jest wielomodalny,
- ewaluacje można równoleglic,
- wystarczy dobre rozwiązanie, niekoniecznie dowód optymalności.

Metody gradientowe/quasi-Newtonowskie warto wybrać, gdy:

- funkcja jest gładka,
- gradient jest dostępny,
- problem jest wypukły albo lokalna dokładność jest ważna,
- wymiar jest duży,
- liczba ewaluacji funkcji celu ma być mała.

Często stosuje się hybrydy: algorytm ewolucyjny znajduje obiecujący region, a metoda lokalna dopracowuje rozwiązanie. Takie podejście nazywa się czasem algorytmem memetycznym.

5.2.7 Co powiedzieć na obronie

Algorytm ewolucyjny jest globalną, losową, populacyjną heurystyką bez wymogu gradientu, dobrą dla funkcji trudnych, niégładkich i wielomodalnych. Metody gradientowe, Nelder-Mead i quasi-Newton są bardziej lokalne; dla problemów wypukłych i gładkich dają znacznie lepsze gwarancje oraz dokładność. Ewolucja jest bardziej uniwersalna, ale kosztowna i mniej przewidywalna; metody wypukłe są szybsze i lepiej uzasadnione, gdy spełnione są ich założenia.

5.3 Metody symulacji Monte Carlo w grach

Metody Monte Carlo w grach używają losowych symulacji rozgrywki do oceny ruchów. Są szczególnie użyteczne, gdy przestrzeń stanów jest ogromna, trudno zbudować dobrą funkcję oceny, a pełne przeszukiwanie minimax jest niemożliwe.

5.3.1 Podstawowa idea

Dla danego stanu gry i możliwego ruchu wykonujemy wiele losowych dogrywek do końca gry albo do ustalonej głębokości. Wynik ruchu oceniamy średnią nagrodą:

$$\hat{V}(a) = \frac{1}{N} \sum_{i=1}^N z_i,$$

gdzie z_i jest wynikiem i -tej symulacji po ruchu a . Ruch z największą średnią jest wybierany.

Zaletą jest prostota i brak konieczności ręcznego projektowania skomplikowanej heurystyki. Wadą jest wariancja i koszt wielu symulacji.

5.3.2 Monte Carlo Tree Search

MCTS łączy symulacje Monte Carlo z budową drzewa przeszukiwania. Najczęściej opisuje się cztery fazy:

1. selekcja - przechodzimy od korzenia do liścia, wybierając dzieci według reguły równoważającej eksplorację i eksploatację,
2. ekspansja - dodajemy nowy węzeł,
3. symulacja - wykonujemy dogrywkę, np. losową albo heurystyczną,
4. propagacja wsteczna - aktualizujemy statystyki odwiedzonych węzłów.

W każdym węźle przechowuje się zwykle liczbę wizyt $N(s)$ i sumę albo średnią nagród $Q(s, a)$.

5.3.3 UCT

Popularną regułą selekcji jest UCT, oparta na upper confidence bound:

$$a = \arg \max_a \left(\bar{X}(s, a) + C \sqrt{\frac{\ln N(s)}{N(s, a)}} \right).$$

Pierwszy składnik promuje ruchy o wysokiej średniej wygranej, drugi promuje ruchy rzadko badane. Parametr C kontroluje eksplorację.

5.3.4 Zastosowania

MCTS był przełomowy w grach takich jak Go, gdzie klasyczne minimax z ręczną funkcją oceny długo było niewystarczające. Stosuje się go też w grach planszowych, grach z niepełną informacją w wariantach determinization albo information set MCTS, planowaniu robotów, grach wideo i problemach decyzyjnych.

5.3.5 Ulepszenia

- rollout policy zamiast całkowicie losowych dogrywek,
- progressive widening dla dużej liczby akcji,
- transposition tables,
- RAVE/AMAF,
- priory z sieci neuronowej,
- value network zastępująca część symulacji,
- parallel MCTS.

AlphaGo i AlphaZero łączyły MCTS z sieciami neuronowymi: sieć polityki sugerowała obiecujące ruchy, a sieć wartości oceniała pozycje.

5.3.6 Co powiedzieć na obronie

Monte Carlo w grach ocenia ruchy przez losowe symulacje. MCTS buduje asymetryczne drzewo tam, gdzie symulacje wskazują obiecujące warianty. Kluczowe fazy to selekcja, ekspansja, symulacja i backpropagation. UCT równoważy eksploatację ruchów dobrych i eksplorację słabo poznanych. Metody te są skuteczne przy ogromnych drzewach gry i słabych heurystykach.

5.4 Pojęcie agenta w SI: opis formalny i własności

Agent w sztucznej inteligencji to system, który postrzega środowisko przez sensory i oddziałuje na nie przez aktyatory. Formalnie agent realizuje odwzorowanie historii percepcji na akcje:

$$f : P^* \rightarrow A,$$

gdzie P^* jest zbiorem możliwych historii perceptorów, a A zbiorem akcji. Często zamiast historii używa się stanu wewnętrznego, który jest jej skompresowaną reprezentacją.

5.4.1 Agent i środowisko

Cykl działania:

percepcja $\rightarrow$ aktualizacja stanu $\rightarrow$ wybór akcji $\rightarrow$ zmiana środowiska.

Środowisko może być:

- w pełni obserwowalne albo częściowo obserwowalne,
- deterministyczne albo stochastyczne,
- epizodyczne albo sekwencyjne,
- statyczne albo dynamiczne,
- dyskretne albo ciągłe,
- jednoagentowe albo wieloagentowe,
- znane albo nieznane.

5.4.2 Racjonalność

Agent racjonalny wybiera akcję maksymalizującą oczekiwaną miarę skuteczności na podstawie dostępnej wiedzy i perceptów. Racjonalność nie oznacza wszechwiedzy. Agent może podjąć decyzję błędną ex post, jeśli była najlepsza ex ante przy dostępnej informacji.

Formalnie można mówić o maksymalizacji oczekiwanej użyteczności:

$$a^* = \arg \max_a \mathbb{E}[U(\text{wynik}) \mid \text{historia}, a].$$

5.4.3 Typy agentów

Prosty agent reaktywny wybiera akcję na podstawie bieżącej percepcji, reguł typu warunek-akcja.

Agent ze stanem utrzymuje stan wewnętrzny opisujący nieobserwowalne aspekty świata.

Agent celowy wybiera akcje prowadzące do celu.

Agent użytecznościowy porównuje różne wyniki przez funkcję użyteczności.

Agent uczący się poprawia działanie na podstawie doświadczenia.

5.4.4 Własności agentów

W systemach wieloagentowych często wymienia się:

- autonomię - agent działa bez bezpośredniego sterowania,
- reaktywność - reaguje na zmiany środowiska,
- proaktywność - inicjuje działania dla osiągnięcia celów,
- społeczność - komunikuje się i współpracuje z innymi agentami,
- adaptacyjność - zmienia zachowanie na podstawie doświadczenia,
- racjonalność - dobiera akcje zgodnie z miarą skuteczności,
- ograniczoną racjonalność - działa przy ograniczonych zasobach obliczeniowych.

5.4.5 Agent w uczeniu ze wzmocnieniem

W RL agent w stanie S_t wybiera akcję A_t , środowisko zwraca nagrodę R_{t+1} i kolejny stan S_{t+1} . Celem jest maksymalizacja oczekiwanego zdyskontowanego zwrotu:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Polityka $\pi(a|s)$ opisuje zachowanie agenta.

5.4.6 Co powiedzieć na obronie

Agent to system postrzegający środowisko i wykonujący akcje. Formalnie można go opisać jako funkcję z historii perceptów do akcji. Agent racjonalny maksymalizuje oczekiwaną miarę skuteczności przy dostępnej wiedzy. Ważne własności to autonomia, reaktywność, proaktywność, zdolność komunikacji i uczenie się.

5.5 Inteligencja rojowa: idea i przykłady realizacji

Inteligencja rojowa opisuje rozwiązywanie problemów przez wiele prostych jednostek, które lokalnie oddziałują ze sobą i środowiskiem, a globalnie tworzą złożone, adaptacyjne zachowanie. Inspiracją są mrówki, pszczoły, ptaki, ryby i inne systemy kolektywne.

5.5.1 Cechy systemów rojowych

- brak centralnego sterowania,
- lokalne reguły zachowania,
- komunikacja bezpośrednia albo przez środowisko,
- samoorganizacja,
- redundancja i odporność,
- równoległość,
- emergencja, czyli powstawanie globalnego wzorca z lokalnych interakcji.

Stigmergia oznacza komunikację przez ślady w środowisku. Klasyczny przykład to feromony mrówek.

5.5.2 Ant Colony Optimization

ACO rozwiązuje problemy kombinatoryczne przez sztuczne mrówki budujące rozwiązania krok po kroku. Prawdopodobieństwo wyboru elementu zależy od feromonu i heurystyki lokalnej:

$$p_{ij} \propto \tau_{ij}^{\alpha} \eta_{ij}^{\beta},$$

gdzie τ_{ij} to poziom feromonu, a η_{ij} to atrakcyjność heurystyczna, np. odwrotność odległości w TSP.

Po zbudowaniu rozwiązań feromon jest aktualizowany:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \Delta\tau_{ij}.$$

Parowanie feromonu zapobiega zbyt szybkiemu zablokowaniu na jednym rozwiązaniu.

5.5.3 Particle Swarm Optimization

PSO działa w przestrzeni ciągłej. Cząstka ma pozycję x_i i prędkość v_i . Aktualizacja:

$$v_i \leftarrow \omega v_i + c_1 r_1 (p_i - x_i) + c_2 r_2 (g - x_i),$$

$$x_i \leftarrow x_i + v_i.$$

Składniki oznaczają: bezwładność, przyciąganie do najlepszego własnego punktu p_i i przyciąganie do najlepszego punktu roju g . PSO jest proste i skuteczne dla wielu problemów ciągłych, ale może przedwcześnie zbiegać.

5.5.4 Artificial Bee Colony

ABC inspirowane jest zachowaniem pszczół. Rozwiązania są źródłami pożywienia. Występują pszczoły zatrudnione, obserwatorzy i zwiadowczynie. Dobre źródła są eksploatowane, słabe porzucane, a zwiadowczynie szukają nowych regionów.

5.5.5 Boids i modele stadne

W symulacji stad ptaków lub ryb globalny ruch powstaje z trzech lokalnych reguł:

- separacja - unikaj kolizji,
- alignment - dopasuj kierunek do sąsiadów,
- cohesion - trzymaj się grupy.

To przykład emergencji: z prostych reguł lokalnych powstaje złożone zachowanie zbiorowe.

5.5.6 Zastosowania

- TSP i optymalizacja tras,
- harmonogramowanie,
- dobór parametrów modeli,
- robotyka rojowa,
- routing w sieciach,
- optymalizacja funkcji ciągłych,
- klastrowanie i eksploracja danych.

5.5.7 Co powiedzieć na obronie

Inteligencja rojowa polega na tym, że wiele prostych jednostek działających lokalnie tworzy globalnie inteligentne zachowanie. Nie ma centralnego sterowania, ważna jest samoorganizacja i komunikacja lokalna. Przykłady to ACO dla problemów kombinatorycznych, PSO dla optymalizacji ciągłej, ABC oraz modele stadne.

5.6 Programowanie w logice na przykładzie Prologu

Programowanie logiczne opisuje problem przez fakty i reguły, a obliczenie polega na logicznym wnioskowaniu. Prolog jest klasycznym językiem programowania logicznego opartym na klauzulach Horna, unifikacji i rezolucji SLD.

5.6.1 Fakty, reguły i zapytania

Fakt:

```
parent(jan, anna).  
parent(anna, piotr).
```

Reguła:

```
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```

Zapytanie:

```
?- grandparent(jan, piotr).
```

Regułę czytamy: X jest dziadkiem/babcią Z , jeśli istnieje Y , takie że X jest rodzicem Y i Y jest rodzicem Z .

5.6.2 Semantyka klauzul Horna

Klauzula Horna ma co najwyżej jeden literal pozytywny. Reguła Prologa:

$$H : - B_1, \dots, B_k$$

odpowiada formule:

$$B_1 \wedge \dots \wedge B_k \rightarrow H.$$

Fakt to reguła z pustym ciałem. Zapytanie jest celem, którego Prolog próbuje dowieść.

5.6.3 Unifikacja

Unifikacja dopasowuje termny przez podstawienia. Dla zapytania:

```
?- parent(jan, X).
```

i faktu:

```
parent(jan, anna).
```

unifikator to $\{X/anna\}$. Prolog zwróci $X = anna$.

5.6.4 Rezolucja SLD i backtracking

Prolog bierze pierwszy cel z listy celów, szuka pierwszej pasującej klauzuli, unifikuje, zastępuje cel ciałem reguły i kontynuuje. Jeśli później dojdzie do porażki, cofa się do ostatniego punktu wyboru i próbuje kolejnej klauzuli. To backtracking.

Kolejność reguł i kolejność celów w ciele ma znaczenie operacyjne, mimo że deklaratorywnie mogą być równoważne. Źle ustawiona rekursja może powodować zapętlenie.

5.6.5 Rekursja

Przykład przodka:

```
ancestor(X, Z) :- parent(X, Z).
ancestor(X, Z) :- parent(X, Y), ancestor(Y, Z).
```

Pierwsza reguła jest przypadkiem bazowym, druga rekurencyjnym.

5.6.6 Negacja jako porażka

Prolog często używa negation as failure:

```
not_p(X) :- \+ p(X).
```

Znaczy to: uznaj $\neg p(X)$, jeśli nie udało się dowieść $p(X)$. To nie jest klasyczna negacja logiczna; opiera się na założeniu zamkniętego świata. Jeśli czegoś nie ma w bazie i nie da się tego wywnioskować, traktujemy to jako fałsz.

5.6.7 Cut

Operator cut ‘!’ ogranicza backtracking. Może poprawiać wydajność albo wymuszać deterministyczny wybór, ale osłabia deklaratywną czytelność programu. Cut zielony nie zmienia znaczenia logicznego, tylko przyspiesza. Cut czerwony zmienia odpowiedzi programu i wymaga ostrożności.

5.6.8 Co powiedzieć na obronie

Prolog reprezentuje wiedzę przez fakty i reguły będące klauzulami Horna. Obliczenie polega na dowodzeniu zapytań przez unifikację, rezolucję SLD i backtracking. Program ma charakter deklaracyjny, ale kolejność reguł i celów wpływa na działanie. Ważne pojęcia to unifikacja, rekursja, negacja jako porażka i cut.

5.7 Generowanie reguł minimalnych z bazy wiedzy w systemach eksperckich

System ekspertowy używa bazy wiedzy i mechanizmu wnioskowania do rozwiązywania problemów z danej dziedziny. Wiedza często ma postać reguł:

IF warunki THEN decyzja.

Reguły minimalne są regułami, z których nie można usunąć żadnego warunku bez utraty zdolności rozróżniania decyzji albo bez zmiany pokrycia zgodnego z założonym kryterium.

5.7.1 System informacyjny i decyzyjny

Dane można przedstawić jako tabelę decyzyjną:

$$S = (U, A \cup \{d\}),$$

gdzie U jest zbiorem obiektów, A zbiorem atrybutów warunkowych, a d atrybutem decyzyjnym. Reguła ma postać:

$$(a_1 = v_1) \wedge \dots \wedge (a_k = v_k) \rightarrow (d = c).$$

5.7.2 Minimalność reguły

Reguła jest minimalna, jeśli:

- jest poprawna albo wystarczająco wiarygodna według przyjętej miary,
- konkluzja pozostaje określona przez warunki,
- żaden warunek z poprzednika nie jest redundantny.

Minimalność nie zawsze oznacza najkrótszą możliwą regułę globalnie. Może oznaczać minimalność względem inkluzji zbioru warunków.

5.7.3 Podejście przez zbiory przybliżone

W teorii zbiorów przybliżonych obiekty są nierozróżnialne względem zbioru atrybutów B , jeśli mają te same wartości wszystkich atrybutów z B . Redukt to minimalny podzbiór atrybutów zachowujący zdolność klasyfikacyjną całego zbioru atrybutów. Core to część wspólna wszystkich reduktów, czyli atrybuty konieczne.

Generowanie reguł:

1. zbudować klasy decyzyjne,
2. wyznaczyć relacje nierozróżnialności,
3. znaleźć redukt lokalne albo globalne,
4. utworzyć reguły z minimalnych kombinacji atrybut-wartość,
5. usunąć warunki redundantne,
6. ocenić wsparcie, dokładność i pokrycie.

5.7.4 Macierz rozróżnialności

Dla par obiektów o różnych decyzjach buduje się zbiory atrybutów, na których obiekty się różnią. Aby reguła odróżniała obiekt od obiektów z inną decyzją, musi zawierać co najmniej jeden atrybut z każdego takiego zbioru. Problem sprowadza się do znalezienia minimalnych hitting sets, co może być kosztowne obliczeniowo.

5.7.5 Miary reguł

Dla reguły $r : P \rightarrow C$:

Wsparcie liczba obiektów spełniających P i C .

Pokrycie jaka część klasy decyzyjnej jest pokryta przez regułę.

Dokładność odsetek obiektów spełniających P , które mają decyzję C :

$$acc(r) = \frac{|P \cap C|}{|P|}.$$

Confidence analog dokładności w regułach asocjacyjnych.

Lift porównanie z częstością klasy bazowej.

W danych zaszumionych wymóg stuprocentowej dokładności może prowadzić do zbyt specyficznych reguł. Dopuszcza się reguły przybliżone z minimalnym wsparciem i minimalną dokładnością.

5.7.6 Usuwanie redundancji

Regułę minimalizuje się przez testowanie każdego warunku:

1. usuń warunek,
2. sprawdź, czy reguła nadal spełnia kryterium jakości,
3. jeśli tak, warunek był redundantny,
4. powtarzaj do punktu stałego.

Usuwanie pojedynczych warunków nie zawsze daje globalnie najkrótszą regułę, ale jest praktyczne. Dokładniejsze metody przeszukują przestrzeń podzbiorów warunków.

5.7.7 Związek z drzewami decyzyjnymi

Reguły można generować z drzewa decyzyjnego: każda ścieżka od korzenia do liścia tworzy regułę. Następnie reguły można przycinać, usuwając warunki, które nie pogarszają jakości na danych walidacyjnych. Takie reguły są czytelne, choć zależą od heurystyki budowy drzewa.

5.7.8 Co powiedzieć na obronie

Reguły minimalne to reguły decyzyjne bez zbędnych warunków. Generuje się je z tabeli decyzyjnej przez analizę rozróżnialności obiektów, redukcję atrybutów, zbiory przybliżone albo drzewa decyzyjne. Minimalizacja polega na usuwaniu warunków, które nie są potrzebne do zachowania decyzji lub jakości reguły. Reguły ocenia się przez wsparcie, dokładność i pokrycie.

6 Podstawy przetwarzania danych

6.1 Metody redukcji wymiaru danych

Redukcja wymiaru polega na zastąpieniu danych opisanych wieloma zmiennymi przez reprezentację o mniejszej liczbie wymiarów przy możliwie małej utracie istotnej informacji. Stosuje się ją dla wizualizacji, przyspieszenia algorytmów, usuwania szumu, walki z przekleństwem wymiarowości i poprawy generalizacji.

6.1.1 Redukcja wymiaru a selekcja cech

Redukcja wymiaru tworzy nowe zmienne będące transformacjami cech oryginalnych. Selekcja cech wybiera podzbiór istniejących cech. PCA tworzy nowe składowe liniowe; selekcja może wybrać np. tylko wiek, dochód i liczbę transakcji z oryginalnej tabeli.

6.1.2 PCA

Principal Component Analysis znajduje ortogonalne kierunki maksymalnej wariancji danych. Po centrowaniu macierzy danych X szukamy kierunku w , który maksymalizuje:

$$\text{Var}(Xw)$$

przy ograniczeniu $\|w\| = 1$. Rozwiązaniem są wektory własne macierzy kowariancji:

$$\Sigma = \frac{1}{n-1} X^T X.$$

Pierwsza składowa główna wyjaśnia największą wariancję, druga największą wariancję w kierunku ortogonalnym do pierwszej itd.

Zalety PCA:

- prostota,
- szybkość,
- dekorrelacja zmiennych,
- możliwość interpretacji przez wariancję wyjaśnioną.

Wady:

- liniowość,
- wrażliwość na skalowanie cech,
- skupienie na wariancji, niekoniecznie na informacji predykcyjnej,
- ograniczona interpretowalność składowych.

6.1.3 SVD

Singular Value Decomposition rozkłada macierz:

$$X = U\Sigma V^T.$$

Przycięcie do k największych wartości osobliwych daje najlepsze przybliżenie rzędu k w normie Frobeniusa. SVD jest podstawą PCA i metod latent semantic analysis.

6.1.4 LDA jako redukcja nadzorowana

Linear Discriminant Analysis szuka kierunków dobrze rozdzielających klasy. Maksymalizuje stosunek wariancji międzyklasowej do wewnątrzklasowej. Dla C klas redukuje do co najwyżej $C - 1$ wymiarów. W przeciwieństwie do PCA używa etykiet.

6.1.5 Metody nieliniowe

Kernel PCA wykonuje PCA w przestrzeni cech zadanej przez kernel.

Isomap zachowuje odległości geodezyjne na rozmaitości.

LLE zakłada, że punkt można odtworzyć z lokalnych sąsiadów.

t-SNE służy głównie do wizualizacji, zachowuje lokalne podobieństwa, ale zniekształca globalne odległości.

UMAP szybka metoda wizualizacji i redukcji, często lepiej zachowuje strukturę globalną niż t-SNE, ale nadal wymaga ostrożnej interpretacji.

Autoenkodery sieci neuronowe uczące kodu niskowymiarowego przez rekonstrukcję wejścia.

6.1.6 Dobór liczby wymiarów

Można użyć:

- progu wariancji wyjaśnionej, np. 95% w PCA,
- wykresu osypiska,
- walidacji krzyżowej dla jakości modelu predykcyjnego,
- metryk rekonstrukcji,
- stabilności reprezentacji.

6.1.7 Co powiedzieć na obronie

Redukcja wymiaru tworzy niskowymiarową reprezentację danych. PCA jest podstawową metodą liniową maksymalizującą wariancję, SVD daje jej stabilną implementację, LDA używa etykiet, a metody nieliniowe takie jak t-SNE, UMAP i autoenkodery wykrywają bardziej złożone struktury. Trzeba pamiętać o skalowaniu, utracie informacji i walidacji liczby wymiarów.

6.2 Metody selekcji cech

Selekcja cech wybiera podzbiór oryginalnych zmiennych. Celem jest poprawa generalizacji, redukcja kosztu pomiaru, przyspieszenie modelu, interpretowalność i zmniejszenie ryzyka overfittingu.

6.2.1 Filtry

Metody filtrujące oceniają cechy niezależnie od konkretnego modelu albo z użyciem prostych statystyk:

- wariancja cechy,
- korelacja z etykietą,
- test χ^2 dla cech kategorycznych,
- ANOVA F-test,
- mutual information,
- współczynnik korelacji rang Spearmana,
- usuwanie jednej z silnie skorelowanych cech.

Filtry są szybkie i skalowalne, ale mogą ignorować interakcje między cechami.

6.2.2 Wrappery

Wrapper używa modelu jako czarnej skrzynki i ocenia podzbiory cech przez walidację:

- forward selection - zaczynamy od pustego zbioru i dodajemy cechy,
- backward elimination - zaczynamy od wszystkich i usuwamy,
- recursive feature elimination,
- przeszukiwanie heurystyczne, np. algorytm genetyczny.

Wrappery mogą dawać lepsze podzbiory dla konkretnego modelu, ale są kosztowne i podatne na przeuczenie, jeśli walidacja jest źle zorganizowana.

6.2.3 Metody wbudowane

Embedded methods wybierają cechy w trakcie uczenia modelu:

- Lasso, czyli regularyzacja L_1 , zeruje część współczynników,
- Elastic Net łączy L_1 i L_2 ,
- drzewa i lasy losowe dają importance cech,
- gradient boosting ma miary gain/split importance,
- modele liniowe z karami grupowymi wybierają grupy cech.

Trzeba uważać na interpretację importance w drzewach: cechy o wielu możliwych podziałach albo silnie skorelowane mogą być faworyzowane lub dzielić znaczenie.

6.2.4 Stability selection

Stability selection wielokrotnie losuje podpróbki danych, wykonuje selekcję i wybiera cechy pojawiające się stabilnie. Pomaga odróżnić cechy rzeczywiście istotne od przypadkowych, zwłaszcza gdy liczba cech jest duża.

6.2.5 Selekcja a przeciek danych

Selekcja cech musi być wykonywana wewnątrz pętli walidacji. Błąd: wybrać cechy na całym zbiorze, a potem zrobić cross-validation. Informacja ze zbioru testowego przecieka wtedy do modelu i wynik jest zawyżony.

6.2.6 Co powiedzieć na obronie

Selekcja cech wybiera podzbiór oryginalnych zmiennych. Filtry są szybkie i niezależne od modelu, wrappery oceniają podzbiory przez model i walidację, a metody wbudowane wybierają cechy podczas uczenia, np. Lasso albo drzewa. Najważniejsze jest unikanie przecieku danych i ocena stabilności wyboru.

6.3 Metody postępowania z brakami w danych

Braki danych mogą zniekształcić analizę i modelowanie. Wybór metody zależy od mechanizmu braków, udziału braków, typu zmiennej i modelu docelowego.

6.3.1 Mechanizmy braków

MCAR Missing Completely At Random - brak niezależny od danych obserwowanych i nieobserwowanych. Usunięcie wierszy nie wprowadza biasu, ale zmniejsza moc statystyczną.

MAR Missing At Random - brak zależy od danych obserwowanych, ale nie od brakującej wartości po uwzględnieniu obserwowanych zmiennych. Można go modelować.

MNAR Missing Not At Random - brak zależy od samej brakującej wartości albo ukrytej zmiennej. Najtrudniejszy przypadek.

Przykład MNAR: osoby z bardzo wysokim dochodem częściej nie podają dochodu.

6.3.2 Usuwanie danych

- listwise deletion - usunięcie wierszy z jakimkolwiek brakiem,
- pairwise deletion - użycie dostępnych par obserwacji dla danej statystyki,
- usuwanie cech z bardzo dużym udziałem braków.

Usuwanie jest proste, ale może wprowadzać bias i marnować dane, jeśli braki nie są MCAR.

6.3.3 Prosta imputacja

- średnia albo mediana dla zmiennych liczbowych,
- dominanta dla zmiennych kategoriowych,
- stała wartość specjalna, np. "unknown",
- imputacja grupowa, np. mediana w obrębie segmentu.

Mediana jest odporniejsza na odstające wartości niż średnia. Prosta imputacja zaniża wariancję i może osłabić zależności między zmiennymi.

6.3.4 Imputacja modelowa

- k-nearest neighbors imputation,
- regresja predykcyjna,
- drzewa i lasy,
- EM dla modeli probabilistycznych,
- multiple imputation,
- autoenkodery albo modele generatywne.

Multiple imputation tworzy wiele uzupełnionych zbiorów, analizuje każdy i łączy wyniki, uwzględniając niepewność imputacji.

6.3.5 Indykatory braków

Czasem sam fakt braku jest informacyjny. Dodaje się cechę binarną:

$$m_j = \begin{cases} 1, & x_j \text{ było brakujące,} \\ 0, & \text{w przeciwnym razie.} \end{cases}$$

To bywa skuteczne przy MAR/MNAR, ale nie zastępuje analizy mechanizmu braków.

6.3.6 Modele obsługujące braki

Niektóre modele potrafią obsługiwać braki bez jawnej imputacji, np. wybrane implementacje drzew gradientowych mają kierunek domyślny dla braków. Nadal trzeba rozumieć, co model robi z brakami i czy zachowanie jest zgodne z problemem.

6.3.7 Co powiedzieć na obronie

Najpierw trzeba rozpoznać mechanizm braków: MCAR, MAR albo MNAR. Można usuwać wiersze lub cechy, imputować prostymi statystykami, imputować modelowo, stosować multiple imputation albo dodać indykatory braków. Imputację trzeba dopasować tylko na danych treningowych, aby uniknąć przecieku.

6.4 Estymatory błędów

Estymator błędu ocenia, jak dobrze model będzie działał na nowych danych. Błąd treningowy jest zwykle zbyt optymistyczny, bo model był dopasowany do tych danych.

6.4.1 Podział train/validation/test

Najprostszy schemat:

- zbiór treningowy - uczenie parametrów,
- zbiór walidacyjny - dobór hiperparametrów i wybór modelu,
- zbiór testowy - końcowa bezstronna ocena.

Zbiór testowy powinien być użyty tylko raz na końcu. Wielokrotne podejmowanie decyzji na podstawie testu prowadzi do przeuczenia do testu.

6.4.2 Walidacja krzyżowa

W k -fold cross-validation dane dzieli się na k części. Model trenuje się k razy, za każdym razem na $k - 1$ częściach, a testuje na pozostałej. Wyniki uśrednia się:

$$\hat{E}_{CV} = \frac{1}{k} \sum_{i=1}^k E_i.$$

Typowe wartości to $k = 5$ albo $k = 10$.

Leave-one-out CV to przypadek $k = n$. Ma mały bias, ale może mieć dużą wariancję i jest kosztowna.

6.4.3 Stratyfikacja i dane zależne

Dla klasyfikacji niezbalansowanej stosuje się stratified CV, aby proporcje klas były podobne w foldach. Dla szeregów czasowych nie wolno mieszać przyszłości z przeszłością; używa się walidacji kroczącej. Dla danych grupowych trzeba dzielić po grupach, aby obserwacje tej samej osoby, klienta albo urządzenia nie trafiały jednocześnie do treningu i testu.

6.4.4 Bootstrap

Bootstrap losuje próbki z powtórzeniami z danych treningowych. Obserwacje niewylosowane tworzą out-of-bag set. Bootstrap pozwala oceniać niepewność estymatora i stabilność modelu. W lasach losowych OOB error jest naturalnym estymatorem błędu.

6.4.5 Macierz pomyłek i miary klasyfikacji

Dla klasyfikacji binarnej:

- TP - prawdziwie pozytywne,
- TN - prawdziwie negatywne,
- FP - fałszywie pozytywne,
- FN - fałszywie negatywne.

Miary:

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN},$$

$$precision = \frac{TP}{TP + FP},$$

$$recall = \frac{TP}{TP + FN},$$

$$F1 = \frac{2 \cdot precision \cdot recall}{precision + recall}.$$

Accuracy jest mylące przy silnie niezbalansowanych klasach.

6.4.6 Miary regresji

$$MAE = \frac{1}{n} \sum_i |y_i - \hat{y}_i|,$$

$$MSE = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2,$$

$$RMSE = \sqrt{MSE}.$$

MAE jest odporniejsze na outliery, MSE/RMSE mocniej karzą duże błędy. R^2 mierzy część wariancji wyjaśnionej przez model.

6.4.7 Bias i wariancja estymatora błędu

Estymator błędu może być obciążony i mieć wariancję. Mały zbiór testowy daje dużą niepewność. Cross-validation zmniejsza zależność od pojedynczego podziału, ale wyniki foldów nie są całkowicie niezależne. Przy porównywaniu modeli warto stosować testy statystyczne albo przedziały ufności.

6.4.8 Co powiedzieć na obronie

Błąd modelu trzeba estymować na danych nieużytych do uczenia. Podstawowe metody to holdout, train/validation/test, walidacja krzyżowa, bootstrap i OOB. Miara błędu zależy od zadania: accuracy, precision, recall, F1, AUC dla klasyfikacji; MAE, MSE, RMSE dla regresji. Należy unikać przecieku danych i dobierać schemat walidacji do struktury danych.

7 Uczenie ze wzmocnieniem

7.1 Uczenie ze wzmocnieniem: pasywne, aktywne i polityki

Uczenie ze wzmocnieniem bada sytuację, w której agent uczy się przez interakcję ze środowiskiem. Agent wykonuje akcje, obserwuje następne stany i otrzymuje nagrody. Nie dostaje poprawnych etykiet akcji, lecz sygnał oceny skutków działania.

7.1.1 Cykl interakcji

W chwili t agent jest w stanie S_t , wybiera akcję A_t , środowisko zwraca nagrodę R_{t+1} i nowy stan S_{t+1} . Celem jest maksymalizacja oczekiwanego zwrotu:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

gdzie $\gamma \in [0, 1]$ jest współczynnikiem dyskontowym. Dla zadań epizodycznych suma kończy się w stanie terminalnym.

7.1.2 Polityka

Polityka opisuje sposób wyboru akcji:

$$\pi(a|s) = P(A_t = a \mid S_t = s).$$

Polityka deterministyczna przypisuje każdemu stanowi jedną akcję:

$$\pi(s) = a.$$

Polityka stochastyczna przypisuje rozkład prawdopodobieństwa na akcjach. Stochastyczność jest ważna dla eksploracji i dla środowisk częściowo obserwowalnych albo wieloagentowych.

7.1.3 Pasywne uczenie ze wzmocnieniem

W pasywnym RL polityka jest dana. Agent nie decyduje, jak ją ulepszyć, lecz ocenia jej jakość. Typowe zadanie: policy evaluation, czyli estymacja funkcji wartości $V^\pi(s)$ dla ustalonej polityki π .

Przykład: mamy strategię gry w blackjacka "dobieraj do 20, potem stop" i chcemy oszacować wartość stanów przy tej strategii. Metody: Monte Carlo prediction, TD(0), dynamic programming, jeśli model jest znany.

7.1.4 Aktywne uczenie ze wzmocnieniem

W aktywnym RL agent ma znaleźć dobrą albo optymalną politykę. Musi jednocześnie:

- eksplorować, aby poznać skutki akcji,
- eksploatować, aby wybierać akcje już znane jako dobre.

To problem exploration-exploitation. Strategie:

- ε -greedy - z prawdopodobieństwem $1 - \varepsilon$ wybierz najlepszą znaną akcję, z ε losową,
- softmax/Boltzmann exploration,
- upper confidence bound,
- Thompson sampling,
- dodawanie szumu do parametrów albo akcji.

7.1.5 On-policy i off-policy

W uczeniu on-policy uczymy wartość albo poprawiamy tę samą politykę, która generuje dane. Przykład: SARSA.

W uczeniu off-policy dane generuje polityka behawioralna b , a uczymy politykę docelową π . Przykład: Q-learning. Off-policy pozwala uczyć politykę zachłanną na danych z polityki eksploracyjnej, ale może być mniej stabilne, zwłaszcza z aproksymacją funkcji.

7.1.6 Funkcje wartości

Wartość stanu:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

Wartość akcji:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Funkcja V mówi, jak dobry jest stan przy danej polityce. Funkcja Q mówi, jak dobra jest akcja w stanie, dlatego jest szczególnie użyteczna, gdy nie znamy modelu przejść.

7.1.7 Co powiedzieć na obronie

RL polega na uczeniu przez interakcję: stan, akcja, nagroda, kolejny stan. Polityka mapuje stany na akcje albo rozkłady akcji. Pasywne RL ocenia zadaną politykę, aktywne RL szuka polityki optymalnej i musi rozwiązać dylemat eksploracji i eksploatacji. Wartości V^π i Q^π opisują oczekiwany zwrot.

7.2 Decyzyjny proces Markowa: definicja i przykłady

MDP jest formalnym modelem sekwencyjnego podejmowania decyzji przy niepewności. Zakłada własność Markowa: przyszłość zależy od bieżącego stanu i akcji, a nie od pełnej historii.

7.2.1 Definicja

Skończony MDP można zapisać jako

$$M = (S, A, p, r, \gamma),$$

gdzie:

- S - zbiór stanów,
- A - zbiór akcji,
- $p(s', r|s, a)$ - prawdopodobieństwo przejścia do stanu s' i otrzymania nagrody r po wykonaniu akcji a w stanie s ,
- r - funkcja nagrody, często jako wartość oczekiwana,
- $\gamma \in [0, 1]$ - współczynnik dyskontowy.

Warunek normalizacji:

$$\sum_{s' \in S} \sum_{r \in R} p(s', r|s, a) = 1 \quad \text{dla każdego } s, a.$$

Z rozkładu $p(s', r|s, a)$ można wyznaczyć prawdopodobieństwo przejścia:

$$p(s'|s, a) = \sum_r p(s', r|s, a),$$

oraz oczekiwaną nagrodę:

$$r(s, a) = \sum_r \sum_{s'} r p(s', r|s, a).$$

7.2.2 Własność Markowa

Proces spełnia własność Markowa, jeśli:

$$P(S_{t+1}, R_{t+1} | S_0, A_0, \dots, S_t, A_t) = P(S_{t+1}, R_{t+1} | S_t, A_t).$$

Stan powinien więc zawierać całą informację z historii potrzebną do przewidywania przyszłości. Jeśli obserwacja nie jest pełnym stanem, mamy POMDP, czyli częściowo obserwowalny MDP.

7.2.3 Równania Bellmana

Dla ustalonej polityki:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V^\pi(s')].$$

Dla funkcji akcji:

$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a') \right].$$

Optymalna funkcja wartości spełnia:

$$V^*(s) = \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')].$$

Optymalna funkcja akcji:

$$Q^*(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \max_{a'} Q^*(s',a') \right].$$

7.2.4 Przykłady MDP

Recycling robot stan to poziom baterii, np. high/low. Akcje to search, wait, recharge. Nagrody zależą od zebrania śmieci i kar za rozładowanie.

Gridworld stan to pozycja agenta na kratownicy, akcje to ruchy, przejścia mogą być stochastyczne, a nagrody przypisane do pól terminalnych i kosztów ruchu.

Blackjack stan opisuje sumę kart gracza, odkrytą kartę krupiera i posiadanie używalnego asa. Akcje to hit/stand, nagroda końcowa to wygrana, przegrana albo remis.

Cart-pole stan obejmuje pozycję wózka, prędkość, kąt słupka i prędkość kątową. Akcje to siła w lewo-/prawo, nagroda za utrzymanie równowagi.

Bioreaktor stan to odczyty sensorów i skład, akcje to ustawienia temperatury i mieszania, nagroda to tempo produkcji pożądanej substancji.

7.2.5 Planowanie i uczenie

Jeśli model p i r jest znany, możemy planować metodami programowania dynamicznego: policy evaluation, policy iteration, value iteration. Jeśli model nie jest znany, agent uczy się z doświadczenia metodami Monte Carlo, TD, SARSA, Q-learning albo metodami z aproksymacją funkcji.

7.2.6 Co powiedzieć na obronie

MDP to model (S, A, p, r, γ) , w którym przyszłość zależy tylko od bieżącego stanu i akcji. Polityka wybiera akcje, funkcje V^π i Q^π opisują oczekiwany zwrot, a równania Bellmana wiążą wartość stanu z nagrodą i wartością następnych stanów. Przykłady to robot sprząający, gridworld, blackjack, cart-pole i sterowanie procesem.

7.3 Metoda różnic czasowych TD-learning: idea, zakres stosowania, wady i zalety oraz porównanie z Monte Carlo

Temporal Difference learning łączy idee Monte Carlo i programowania dynamicznego. Tak jak Monte Carlo uczy się z doświadczenia, bez pełnego modelu środowiska. Tak jak programowanie dynamiczne używa bootstrappingu, czyli aktualizuje wartość na podstawie obecnego oszacowania wartości następnego stanu.

7.3.1 TD(0)

Dla ustalonej polityki π aktualizacja TD(0) ma postać:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)].$$

Wyrażenie

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

to błąd TD.

Cel aktualizacji TD to:

$$R_{t+1} + \gamma V(S_{t+1}),$$

czyli jednokrokowe przybliżenie zwrotu. W Monte Carlo celem jest pełny zwrot G_t po zakończeniu epizodu.

7.3.2 Algorytm predykcji TD(0)

1. Zainicjalizuj $V(s)$ dowolnie.
2. Dla każdego epizodu:
 - (a) rozpocznij w stanie S ,
 - (b) wybierz akcję zgodnie z polityką π ,
 - (c) wykonaj akcję, obserwuj R i S' ,
 - (d) zaktualizuj

$$V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)],$$
 - (e) ustaw $S \leftarrow S'$,
 - (f) powtarzaj do stanu terminalnego.

7.3.3 Porównanie TD i Monte Carlo

Cecha	Monte Carlo	TD
Model środowiska	nie wymaga	nie wymaga
Moment aktualizacji	po epizodzie	po każdym kroku
Cel aktualizacji	pełny zwrot G_t	$R_{t+1} + \gamma V(S_{t+1})$
Epizodyczność	wymaga epizodów albo specjalnych wariantów	działa w zadaniach ciągłych
Bootstrapping	nie	tak
Bias	mniejszy przy poprawnych próbkach	większy przez bootstrapping
Wariancja	często większa	zwykle mniejsza

Szybkość uczenia online

wolniejsza informacja

szybka aktualizacja

MC jest nieobciążone względem rzeczywistych zwrotów przy próbkowaniu z polityki, ale może mieć dużą wariancję. TD wprowadza bias, bo używa własnych oszacowań, ale zwykle ma mniejszą wariancję i uczy się szybciej w zadaniach sekwencyjnych.

7.3.4 SARSA

SARSA jest on-policy metodą kontroli TD:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

Nazwa pochodzi od sekwencji:

$$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}.$$

Ponieważ używa akcji faktycznie wybranej przez bieżącą politykę, uczy wartości polityki eksploracyjnej.

7.3.5 Q-learning

Q-learning jest off-policy:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right].$$

Uczy wartości polityki zachłannej, nawet jeśli dane są generowane przez politykę eksploracyjną, np. ϵ -greedy.

7.3.6 Zakres stosowania

TD jest użyteczne, gdy:

- środowisko jest nieznane,
- dane przychodzą online,
- epizody są długie albo nie kończą się,
- chcemy aktualizować po każdym kroku,
- pełny zwrot Monte Carlo miałby dużą wariancję.

TD jest podstawą wielu metod RL, także z aproksymacją funkcji i deep RL.

7.3.7 Wady

- bias przez bootstrapping,
- wrażliwość na krok uczenia α ,
- możliwa niestabilność przy aproksymacji funkcji, off-policy i bootstrappingu,
- potrzeba eksploracji dla metod kontroli,
- trudność doboru reprezentacji stanu.

7.3.8 Zalety

- uczenie online,
- brak potrzeby modelu środowiska,
- brak potrzeby czekania do końca epizodu,
- często mniejsza wariancja niż MC,
- możliwość stosowania w zadaniach ciągłych,
- naturalne połączenie z funkcjami wartości akcji.

7.3.9 Co powiedzieć na obronie

TD-learning aktualizuje wartość stanu po każdym kroku, używając różnicy między starą wartością a celem $R + \gamma V(S')$. Błąd TD to $\delta = R + \gamma V(S') - V(S)$. W porównaniu z Monte Carlo TD nie czeka do końca epizodu i zwykle ma mniejszą wariancję, ale jest obciążone przez bootstrapping. SARSA jest on-policy, Q-learning off-policy.

8 Sieci neuronowe

8.1 Uczenie nadzorowane i nienadzorowane: definicje i przykłady

Uczenie sieci neuronowej polega na zmianie wag połączeń tak, aby sieć realizowała pożądaną funkcję. Struktura połączeń i wartości wag określają, jakie przekształcenie wejścia w wyjście wykonuje sieć. W klasycznym podziale wyróżnia się uczenie nadzorowane, nienadzorowane i uczenie ze wzmocnieniem.

8.1.1 Uczenie nadzorowane

W uczeniu nadzorowanym dane treningowe mają postać par:

$$(x^{(i)}, y^{(i)}), \quad i = 1, \dots, n,$$

gdzie $x^{(i)} \in \mathbb{R}^m$ jest wektorem wejściowym, a $y^{(i)} \in \mathbb{R}^k$ pożądanym wyjściem. Zadaniem jest nauczenie funkcji:

$$f_{\theta} : \mathbb{R}^m \rightarrow \mathbb{R}^k$$

minimalizującej błąd predykcji na nowych danych.

Uczenie polega na minimalizacji funkcji straty:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x^{(i)}), y^{(i)}).$$

Dla regresji często stosuje się błąd kwadratowy:

$$L = \frac{1}{2} \sum_i \sum_j (y_j^{(i)} - \hat{y}_j^{(i)})^2.$$

Dla klasyfikacji wieloklasowej z softmaxem typowa jest entropia krzyżowa:

$$CE = - \sum_{j=1}^k y_j \log \hat{y}_j.$$

Przy one-hot encoding sprowadza się do – log prawdopodobieństwa klasy prawdziwej.

Przykłady:

- klasyfikacja obrazów,
- rozpoznawanie cyfr,
- klasyfikacja spamu,
- regresja cen mieszkań,
- prognoza popytu,
- segmentacja obrazu z etykietami pikseli.

8.1.2 Uczenie nienadzorowane

W uczeniu nienadzorowanym mamy tylko wejścia:

$$x^{(1)}, \dots, x^{(n)}$$

bez etykiet. Celem jest odkrycie struktury danych: grup, reprezentacji, czynników zmienności albo rozkładu.

Przykłady zadań:

- klastrowanie,
- redukcja wymiaru,
- detekcja anomalii,
- kompresja reprezentacji,
- modelowanie generatywne,
- samoorganizujące mapy cech.

W sieciach neuronowych przykładami są autoenkodery, Restricted Boltzmann Machines, sieci Kohonena, uczenie Hebbowskie, WTA/WTM oraz modele generatywne.

8.1.3 WTA i WTM

Winner Takes All oznacza, że dla danego wejścia zwycięża neuron o największej aktywacji, a uczeniu podlega głównie on. Winner Takes Most rozszerza to na sąsiedztwo zwycięzcy. WTM jest podstawą samoorganizujących map Kohonena: aktualizuje się zwycięski neuron i jego sąsiadów, co prowadzi do topologicznego uporządkowania mapy.

8.1.4 Porównanie

Cecha	Uczenie nadzorowane	Uczenie nienadzorowane
Dane	wejścia i etykiety	same wejścia
Cel	predykcja znanego celu	odkrycie struktury
Funkcja straty	porównuje predykcję z etykietą	rekonstrukcja, podobieństwo, gęstość, topologia

Przykłady	klasyfikacja, regresja	klastrowanie, PCA, SOM, autoenkodery
Ocena	accuracy, F1, MSE	silhouette, rekonstrukcja, ocena jakości reprezentacji

8.1.5 Co powiedzieć na obronie

Uczenie nadzorowane korzysta z par wejście-wyjście i minimalizuje błąd względem etykiet. Uczenie nie-nadzorowane nie ma etykiet i szuka struktury w danych, np. klastrów albo reprezentacji. Przykładem nadzorowanym jest MLP klasyfikujący cyfry, a nienadzorowanym mapa Kohonena albo autoenkoder.

8.2 Porównanie modeli sieciowych Hopfielda, Grossberga i Kohonena

Modele Hopfielda, Grossberga i Kohonena należą do klasycznych architektur sieci neuronowych, ale rozwiązują inne zadania i mają inne mechanizmy uczenia.

8.2.1 Sieć Hopfielda

Sieć Hopfielda jest rekurencyjną siecią z pełnymi symetrycznymi połączeniami między neuronami i bez połączeń własnych:

$$w_{ij} = w_{ji}, \quad w_{ii} = 0.$$

Klasyczny model dyskretny ma neurony bipolarne $v_i \in \{-1, 1\}$ albo unipolarne $v_i \in \{0, 1\}$. Stan sieci jest wektorem aktywacji. Aktualizacja może być synchroniczna albo asynchroniczna.

Główne zastosowanie: pamięć skojarzeniowa, czyli content-addressable memory. Sieć przechowuje wzorce jako atraktory dynamiki. Po podaniu wzorca zasumionego powinna zbiec do najbliższego zapamiętanego wzorca.

8.2.2 Modele Grossberga

Stephen Grossberg rozwijał modele konkurencyjnego i adaptacyjnego uczenia, w tym Adaptive Resonance Theory. W kontekście klasycznego porównania z Hopfieldem i Kohonenem Grossberg kojarzy się z sieciami:

- uczącymi się stabilnie nowych kategorii,
- opartymi na konkurencji neuronów,
- rozwiązującymi dylemat stability-plasticity,
- używającymi mechanizmu rezonansu między reprezentacją wejścia i kategorią.

ART ma warstwę wejściową i warstwę kategorii. Wejście aktywuje kategorię kandydującą; jeśli podobieństwo przekracza parametr czujności, następuje rezonans i uczenie. Jeśli nie, kategoria jest odrzucana i szukana jest inna albo tworzona nowa. Wysoka czujność tworzy więcej, bardziej szczegółowych kategorii; niska czujność daje kategorie ogólniejsze.

Główne zastosowania: klastrowanie, klasyfikacja inkrementalna, uczenie bez katastroficznego nadpisywania, adaptacyjne rozpoznawanie wzorców.

8.2.3 Sieć Kohonena

Self-Organizing Map jest nienadzorowaną siecią konkurencyjną. Neurony są ułożone zwykle na siatce 1D albo 2D, a każdy neuron ma wektor wag w_i w przestrzeni danych. Dla wejścia x wybiera się best matching unit:

$$c = \arg \min_i \|x - w_i\|.$$

Następnie aktualizuje się zwycięzcę i jego sąsiadów:

$$w_i(t+1) = w_i(t) + \alpha(t)h_{ci}(t)(x - w_i(t)),$$

gdzie h_{ci} maleje z odległością neuronu i od zwycięzcy c na mapie.

SOM tworzy odwzorowanie danych wysokowymiarowych na niskowymiarową siatkę z zachowaniem lokalnej topologii. Służy do wizualizacji, klastrowania i eksploracji danych.

8.2.4 Porównanie

Cecha	Hopfield	Grossberg/ART	Kohonen/SOM
Typ	rekurencyjna pamięć skojarzeniowa	adaptacyjne kategorie, konkurencja	samoorganizująca mapa
Uczenie	często Hebbowskie albo jego modyfikacje	rezonans, dopasowanie, czujność	WTA/WTM, uczenie sąsiedztwa
Zadanie	odtworzenie wzorców, optymalizacja	stabilne uczenie kategorii	klastrowanie i wizualizacja
Dynamika	zbieganie do atraktora	wybór lub tworzenie kategorii	porządkowanie topologiczne
Nadzór	zwykle nienadzorowane-/asocjacyjne	nienadzorowane lub nadzorowane warianty ART	nienadzorowane
Problem kluczowy	pojemność i minima fałszywe	stability-plasticity	dobór mapy, sąsiedztwa i metryki

8.2.5 Co powiedzieć na obronie

Hopfield to rekurencyjna sieć pamięci skojarzeniowej, w której wzorce są atraktorami. Grossberg/ART koncentruje się na adaptacyjnym tworzeniu kategorii i kompromisie stabilność-plastyczność. Kohonen/SOM to nienadzorowana mapa samoorganizująca, która zachowuje topologię danych i nadaje się do klastrowania oraz wizualizacji.

8.3 Dobór architektury sieci neuronowej do wybranego zadania

Dobór architektury powinien wynikać z rodzaju danych, zadania, ograniczeń obliczeniowych i wymagań jakościowych. Nie istnieje jedna najlepsza sieć do wszystkiego.

8.3.1 Klasyfikacja tabularna

Dla danych tabelarycznych często warto zacząć od prostych modeli: regresji logistycznej, drzew, random forest, gradient boosting. Jeśli używamy sieci, typowy jest MLP:

$$x \rightarrow \text{Dense} \rightarrow \text{aktywacja} \rightarrow \dots \rightarrow \text{softmax/sigmoid}.$$

Dla klasyfikacji wieloklasowej ostatnia warstwa ma k neuronów i softmax, a strata to categorical cross-entropy. Dla binarnej - jeden neuron z sigmoidą i binary cross-entropy.

8.3.2 Regresja

Dla regresji używa się MLP z liniowym neuronem wyjściowym albo odpowiednią aktywacją zgodną z zakresem wartości. Strata: MSE, MAE, Huber. Jeśli predykcja ma być nieujemna, można użyć softplus albo modelować logarytm zmiennej.

8.3.3 Obrazy

Dla obrazów naturalnym wyborem są sieci konwolucyjne, bo wykorzystują lokalność i współdzielenie wag. Warstwy konwolucyjne wykrywają lokalne wzorce, głębsze warstwy budują cechy złożone. Dla segmentacji stosuje się architektury encoder-decoder, np. U-Net. Dla klasyfikacji można użyć transfer learningu z modelu wstępnie trenowanego.

8.3.4 Sekwencje i szeregi czasowe

Możliwości:

- RNN, LSTM, GRU - klasyczne modele rekurencyjne,
- 1D CNN - lokalne wzorce w czasie,
- Transformery - długie zależności i równoległe przetwarzanie,
- modele temporal convolutional networks,
- architektury hybrydowe.

Dla predykcji szeregów czasowych ważne są okna czasowe, cechy kalendarzowe, unikanie przecieku przyszłości i walidacja krocząca.

8.3.5 Tekst

Współcześnie standardem są transformery i embeddingi. Dla prostych zadań można użyć bag-of-words z modelem liniowym albo małego MLP. Dla zadań semantycznych używa się modeli językowych, fine-tuningu albo klasyfikacji na embeddingach.

8.3.6 Uczenie nienadzorowane i kompresja

Dla kompresji i reprezentacji stosuje się autoenkodery. Bottleneck wymusza kod niskowymiarowy. Dla wizualizacji klasyczne są SOM, PCA, t-SNE, UMAP. Dla danych obrazowych autoenkoder konwolucyjny zachowuje strukturę przestrzenną.

8.3.7 Pamięć skojarzeniowa i odtwarzanie wzorców

Dla zadania "po zaszumionym wzorcu odtwórz najbliższy zapamiętany wzorec" naturalna jest sieć Hopfielda. Dla dużych i złożonych wzorców klasyczny Hopfield ma ograniczoną pojemność, więc stosuje się modyfikacje reguł uczenia albo nowocześniejsze pamięci asocjacyjne.

8.3.8 Kryteria doboru rozmiaru

Za mała sieć ma duży bias i underfitting. Za duża może mieć overfitting, choć przy dużych danych i regularyzacji duże sieci mogą generalizować dobrze. Dobór obejmuje:

- liczbę warstw,
- liczbę neuronów/kanałów,
- funkcje aktywacji,
- normalizację,
- dropout,
- regularyzację wag,
- learning rate i optimizer,
- batch size,
- early stopping.

8.3.9 Walidacja architektury

Architekturę dobiera się eksperymentalnie na zbiorze walidacyjnym. Należy porównywać z baseline'ami, analizować krzywe uczenia, sprawdzać overfitting i wykonywać ablation study. Dla małych danych często lepszy jest transfer learning albo prostszy model.

8.3.10 Co powiedzieć na obronie

Architekturę dobiera się do struktury danych i zadania: MLP dla danych wektorowych, CNN dla obrazów, RNN/LSTM/Transformer dla sekwencji, autoenkoder dla kompresji, SOM dla mapowania nienadzorowanego, Hopfield dla pamięci skojarzeniowej. Dobór rozmiaru i regularyzacji trzeba potwierdzić walidacją.

8.4 Idea i własności pamięci skojarzeniowych Hopfielda

Pamięć skojarzeniowa Hopfielda adresuje zawartością, nie adresem. Oznacza to, że po podaniu niepełnego albo zaszumionego wzorca sieć powinna odtworzyć najbliższy zapamiętany wzorzec.

8.4.1 Architektura

Klasyczna dyskretna sieć Hopfielda:

- ma N neuronów,
- każdy neuron jest połączony z każdym innym,
- wagi są symetryczne: $w_{ij} = w_{ji}$,
- brak połączeń własnych: $w_{ii} = 0$,
- stan neuronu jest bipolarny $v_i \in \{-1, 1\}$.

Wejście neuronu:

$$u_i = \sum_{j=1}^N w_{ij} v_j - \theta_i,$$

a aktualizacja:

$$v_i \leftarrow \text{sgn}(u_i).$$

8.4.2 Uczenie Hebbowskie

Dla M wzorców bipolarnych $x^1, \dots, x^M, x^s \in \{-1, 1\}^N$, klasyczna reguła Hebba:

$$w_{ij} = \begin{cases} 0, & i = j, \\ \frac{1}{N} \sum_{s=1}^M x_i^s x_j^s, & i \neq j. \end{cases}$$

Macierzowo:

$$W = \frac{1}{N}(XX^T - MI)$$

przy odpowiedniej konwencji zapisu wzorców.

Reguła wzmacnia połączenia neuronów aktywnych razem i osłabia połączenia aktywnych przeciwnie.

8.4.3 Faza rozpoznawania

Podajemy wektor startowy z , np. zaszumiony wzorec. Sieć iteracyjnie aktualizuje neurony. Możliwe wyniki:

- zbieżność do zapamiętanego wzorca,
- zbieżność do wzorca fałszywego,
- w trybie synchronicznym cykl długości 2,
- błędne odtworzenie przy zbyt dużym szumie albo przeciążeniu pamięci.

8.4.4 Funkcja energii

Dla symetrycznych wag i braku wag własnych definiuje się energię:

$$E(v) = -\frac{1}{2} \sum_{i \neq j} w_{ij} v_i v_j + \sum_i \theta_i v_i.$$

Przy aktualizacji asynchronicznej energia nie rośnie. Ponieważ liczba stanów jest skończona, sieć zbiega do punktu stałego. To podstawowa własność stabilizująca model.

8.4.5 Atraktory i baseny przyciągania

Zapamiętane wzorce powinny być minimami lokalnymi energii. Basen przyciągania wzorca to zbiór stanów startowych, z których dynamika zbiega do tego wzorca. Im większy basen, tym większa odporność na szum. Wzorce zbyt podobne do siebie albo zbyt liczne zmniejszają baseny i zwiększają liczbę atraktorów fałszywych.

8.4.6 Pojemność

Dla losowych niezależnych wzorców bipolarnych klasyczna praktyczna estymacja pojemności reguły Hebba wynosi około:

$$M \approx 0.15N.$$

Asymptotyczne wyniki często podają skalowanie rzędu:

$$M \sim \frac{N}{4 \ln N}$$

dla określonych warunków bezbłędnego odtwarzania. Pojemność zależy od definicji dopuszczalnego błędu, korelacji wzorców i reguły uczenia.

8.4.7 Modyfikacje

Wady Hebba: ograniczona pojemność, wzorce fałszywe, problemy dla wzorców skorelowanych i rzadkich. Modyfikacje:

- reguła pseudoodwrotna/projection rule,
- iteracyjne uczenie macierzy wag,
- dobór progów indywidualnych,
- preprocessing wzorców,
- uogólnione sieci Hopfielda.

Zwykle poprawa pojemności kosztuje utratę lokalności albo prostoty biologicznej interpretacji.

8.4.8 Co powiedzieć na obronie

Sieć Hopfielda jest rekurencyjną pamięcią skojarzeniową. Wzorce są zapisywane w wagach, często regułą Hebba, i odpowiadają atraktorom funkcji energii. Po podaniu zaszumionego wzorca sieć przez iteracyjne aktualizacje zbiega do lokalnego minimum energii. W trybie asynchronicznym przy wagach symetrycznych i zerowej przekątnej energia nie rośnie, więc sieć zbiega. Ograniczenia to pojemność, atraktory fałszywe i wrażliwość na korelacje wzorców.

8.5 Algorytm propagacji wstecznej błędu, własności i podstawowe modyfikacje

Backpropagation jest podstawowym algorytmem obliczania gradientów w wielowarstwowych sieciach neuronowych. Łączy przejście w przód, obliczenie straty i propagację gradientu od wyjścia do wejścia za pomocą reguły łańcuchowej.

8.5.1 Przejście w przód

Dla warstwy l :

$$\begin{aligned} z^{[l]} &= W^{[l]} a^{[l-1]} + b^{[l]}, \\ a^{[l]} &= g^{[l]}(z^{[l]}). \end{aligned}$$

Na końcu otrzymujemy predykcję $\hat{y} = a^{[L]}$ i stratę $L(\hat{y}, y)$.

8.5.2 Przejście wstecz

Backpropagation oblicza pochodne straty względem parametrów. Dla warstwy wyjściowej wyznacza się błąd $\delta^{[L]}$, a następnie dla warstw wcześniejszych:

$$\delta^{[l]} = \left((W^{[l+1]})^T \delta^{[l+1]} \right) \odot g'^{[l]}(z^{[l]}).$$

Gradyenty:

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]}(a^{[l-1]})^T, \quad \frac{\partial L}{\partial b^{[l]}} = \delta^{[l]}.$$

Wagi aktualizuje się np. gradient descent:

$$W \leftarrow W - \eta \nabla_W L.$$

8.5.3 Tryby uczenia

Online/stochastic aktualizacja po pojedynczym przykładzie.

Mini-batch aktualizacja po małej paczce przykładów; standard praktyczny.

Batch aktualizacja po całym zbiorze treningowym.

Epoka oznacza jedno przejście przez cały zbiór treningowy. Iteracja to pojedyncza aktualizacja parametrów.

8.5.4 Warunki stopu

- maksymalna liczba epok,
- strata poniżej progu,
- norma aktualizacji poniżej progu,
- brak poprawy na walidacji,
- early stopping.

8.5.5 Zalety backpropagation

- uniwersalność dla różniczkowalnych architektur,
- efektywne obliczanie gradientów przez reuse wyników pośrednich,
- możliwość trenowania MLP i głębokich sieci,
- zgodność z wieloma funkcjami strat,
- łatwe połączenie z optymalizatorami.

8.5.6 Problemy

Z wykładów istotne problemy:

- minima lokalne i punkty siodłowe,
- wolna zbieżność w płaskich obszarach funkcji błędu,
- oscylacje przy zbyt dużym kroku,
- koszt obliczeniowy,
- zanikające i eksplodujące gradienty,
- catastrophic interference/forgetting,
- dobór architektury.

8.5.7 Momentum

Momentum dodaje bezwładność:

$$\Delta w(t) = -\eta \nabla E(w(t)) + \alpha \Delta w(t-1).$$

Pomaga ograniczać oscylacje i przyspieszać ruch w długich płaskich dolinach funkcji błędu. Zbyt duże momentum może powodować niestabilność.

8.5.8 Adaptacyjny learning rate

Jeśli znak pochodnej cząstkowej dla danej wagi pozostaje taki sam, można zwiększać krok; jeśli znak się zmienia, prawdopodobnie przeskoczyliśmy minimum i krok trzeba zmniejszyć. To idea metod takich jak Silva-Almeida, Delta-bar-delta i Rprop.

Rprop używa głównie znaku gradientu, a nie jego wartości:

$$\Delta w_i = -\eta_i \operatorname{sgn} \left(\frac{\partial E}{\partial w_i} \right),$$

przy adaptacji η_i zależnej od zmian znaku gradientu. Jest metodą batchową i dobrze radzi sobie z płaskimi regionami.

8.5.9 QuickProp i metody drugiego rzędu

Metody drugiego rzędu wykorzystują krzywiznę funkcji błędu. Newton:

$$w_{k+1} = w_k - (\nabla^2 E(w_k))^{-1} \nabla E(w_k).$$

Jest szybki lokalnie, ale Hessian i jego odwrotność są kosztowne. QuickProp aproksymuje informację drugiego rzędu dla pojedynczych wag i może przyspieszać uczenie, ale wymaga ostrożności.

8.5.10 Regularyzacja i stabilizacja

Podstawowe modyfikacje współczesne:

- weight decay,
- dropout,
- batch normalization/layer normalization,
- data augmentation,
- early stopping,
- gradient clipping,
- lepsze inicjalizacje, np. Xavier/He,
- ReLU i jej warianty zamiast sigmoid w głębokich sieciach,
- Adam/AdamW.

8.5.11 Co powiedzieć na obronie

Backpropagation oblicza gradient funkcji straty względem wag przez przejście w przód i propagację błędu wstecz regułą łańcuchową. Wagi aktualizuje się gradientowo. Zalety to efektywność i uniwersalność; problemy to minima lokalne, wolna zbieżność, oscylacje, zanik gradientu i katastroficzne zapominanie. Modyfikacje obejmują momentum, adaptacyjne kroki, Rprop, QuickProp, regularyzację, dropout i early stopping.

8.6 Problem katastroficznego zapominania: na czym polega i jak przeciwdziałać

Katastroficzne zapominanie, nazywane też catastrophic interference, polega na tym, że sieć uczona sekwencyjnie nowych zadań albo klas traci wcześniej nabytą wiedzę. Nowe aktualizacje wag nadpisują reprezentacje potrzebne dla starych zadań.

8.6.1 Dlaczego występuje

W klasycznym uczeniu backpropagation wszystkie zadania współdzielą te same parametry. Jeśli uczymy najpierw zadania A, a potem tylko zadania B, gradient z B zmienia wagi bez informacji o tym, że część wag była istotna dla A. W efekcie spada jakość na A.

Problem jest szczególnie silny, gdy:

- dane przychodzą sekwencyjnie,
- nie można przechowywać starych danych,
- zadania korzystają z tych samych parametrów,
- rozkłady danych zmieniają się,
- model ma ograniczoną pojemność albo zbyt agresywne uczenie.

8.6.2 Stability-plasticity dilemma

Model powinien być plastyczny, aby uczyć się nowych informacji, ale stabilny, aby nie tracić starych. Zbyt duża plastyczność powoduje zapominanie. Zbyt duża stabilność blokuje uczenie nowych zadań.

8.6.3 Interleaved learning i rehearsal

Najprostsza metoda to mieszać stare i nowe przykłady. Rehearsal przechowuje bufor reprezentatywnych starych danych i podczas uczenia nowych zadań powtarza stare. Warianty:

- experience replay,
- exemplar memory,
- generative replay - starych danych dostarcza model generatywny,
- pseudo-rehearsal - generowanie syntetycznych przykładów.

Wada: potrzeba przechowywania danych albo generatora, koszt i kwestie prywatności.

8.6.4 Regularyzacja ważnych wag

Metody takie jak Elastic Weight Consolidation ograniczają zmianę wag ważnych dla poprzednich zadań:

$$L(\theta) = L_{new}(\theta) + \lambda \sum_i F_i(\theta_i - \theta_i^*)^2,$$

gdzie F_i mierzy ważność parametru, np. przez informację Fishera. Podobną ideę mają Synaptic Intelligence i Memory Aware Synapses.

8.6.5 Metody modularne

Można przydzielać różne moduły albo części sieci do różnych zadań:

- osobne głowy wyjściowe,
- progressive networks,
- adaptory,
- maski parametrów,
- dynamiczne rozszerzanie architektury.

Zaletą jest ochrona starej wiedzy. Wadą jest rosnący model i problem łączenia wyników modułów.

8.6.6 Zamrażanie i współdzielenie cech

W wykładach pojawia się idea incremental class learning: uczyć klasy kolejno, wyznaczać neurony reprezentujące istotne cechy klasy, zamrażać ich wagi i wykorzystywać je przy kolejnych klasach. Jeśli nowa klasa może użyć już zamrożonych cech, uczenie jest szybsze i mniej niszczy starą wiedzę. To podejście łączy stabilność zamrożonych reprezentacji z plastycznością niezamrożonych części sieci.

8.6.7 Knowledge distillation

Przy uczeniu nowego zadania można karać odchylenie odpowiedzi nowego modelu od odpowiedzi starego modelu na starych albo reprezentatywnych danych. Dzięki temu model zachowuje funkcję wejście-wyjście poprzedniej wersji.

8.6.8 Architektury i reprezentacje

Zapominanie można ograniczać przez:

- większą pojemność modelu,
- reprezentacje disentangled,
- sparse activations,
- normalizację i małe kroki uczenia,
- task-specific adapters,
- metauczenie strategii ciągłego uczenia.

8.6.9 Ocena ciągłego uczenia

Trzeba mierzyć nie tylko wynik na ostatnim zadaniu, ale też:

- średnią jakość na wszystkich zadaniach,
- forgetting measure,
- forward transfer - czy stare uczenie pomaga nowemu,
- backward transfer - czy nowe uczenie poprawia albo pogarsza stare,
- pamięć i koszt obliczeniowy.

8.6.10 Co powiedzieć na obronie

Katastroficzne zapominanie polega na utracie wcześniej nauczanej wiedzy podczas sekwencyjnego uczenia nowych zadań, bo te same wagi są modyfikowane przez nowe gradienty. Przeciwdziała się przez rehearsal, generative replay, regularyzację ważnych wag, modularność, zamrażanie i współdzielenie cech, dynamiczne rozszerzanie architektury oraz distillation. Sednem jest kompromis między stabilnością starej wiedzy i plastycznością potrzebną do uczenia nowej.